

21CSC206T ARTIFICIAL INTELLIGENCE

Unit-II

What is STACK?



Diagram Reference : <http://www.firmcodes.com/write-a-c-program-to-implement-a-stack-using-an-array-and-linked-list/>

- Stack works on “**Last in First out**” or “**First in Last Out**”, principle.
- Plates placed one over another
- The plate at the top is removed first & the bottommost plate kept longest period of time.
- So, it follows LIFO/FILO order.

Stack Example

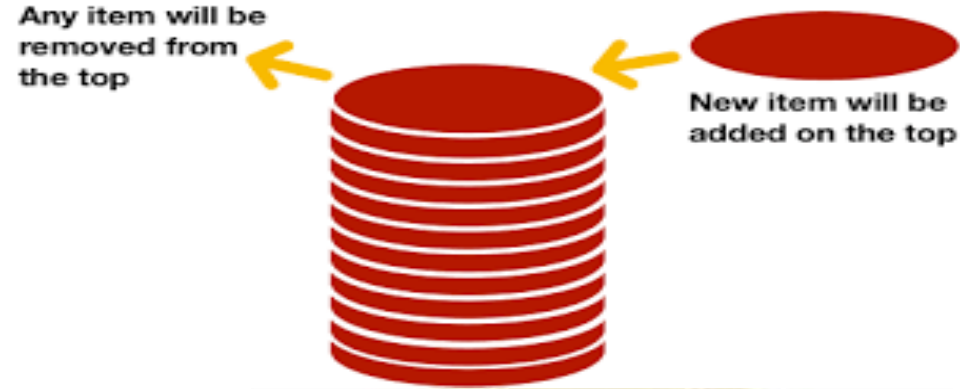


Diagram Reference: <https://www.codesdope.com/course/data-structures-stacks/>

- Always new items are added at top of the stack and also removed from the top of the stack only.
- Entry and Exit at same point

STACK – Data Structure

- Stack is a **Linear Data Structure**
- Follows a particular order to perform the operations.
- The order are either

LIFO (Last In First Out)

OR

FIFO (First In Last Out).

STACK - Operations

TWO PRIMARY OPERATIONS

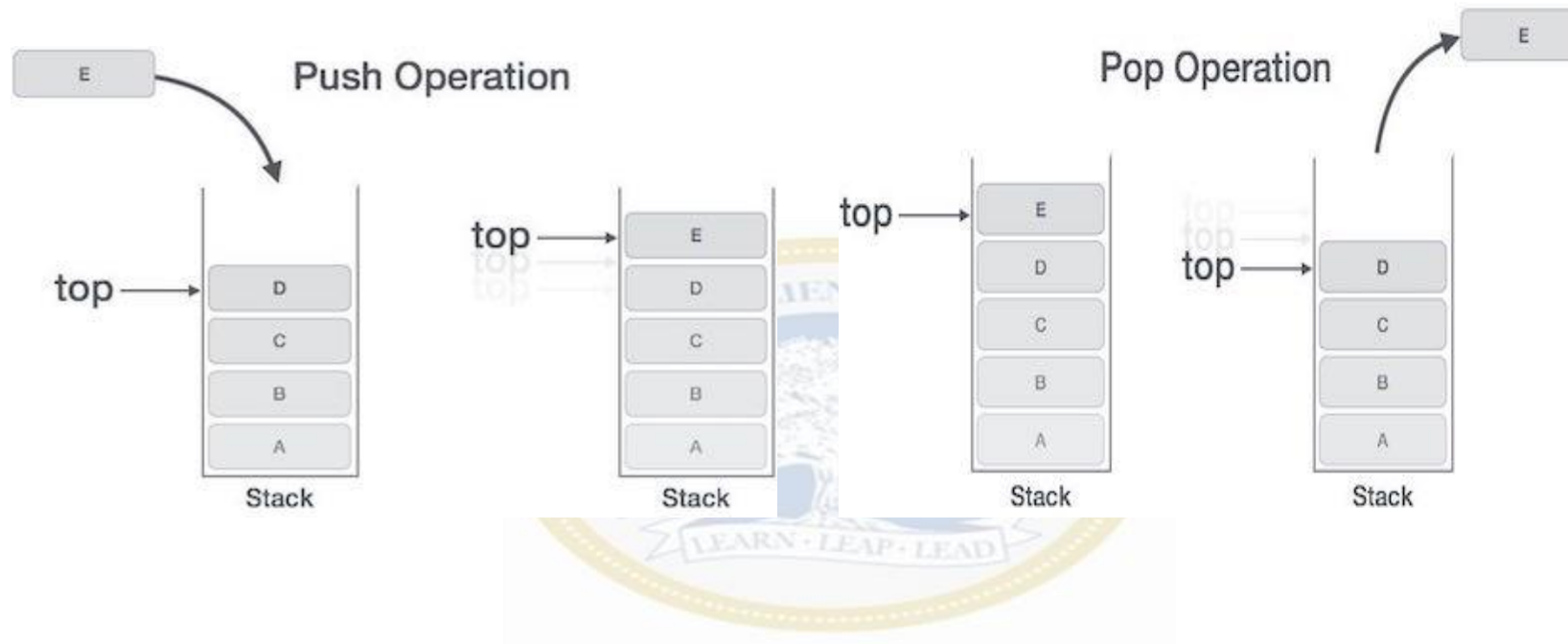
- **push**
 - Pushing (storing) an element on the stack
 - If the stack is full, Overflow condition is enabled.
- **Pop**
 - Removing (accessing) an element from the stack.
 - The elements are popped in the decreasing order.
 - If the stack is empty, Underflow condition enabled.

STACK – Additional Functionality

- For effective utilization of the stack, Status of the stack should be checked. For that additional functionality is of the stack is given below
- **Peek or Top**
 - Accessing the top element. Without removing the top element.
- **isFull**
 - check if stack is full.
- **isEmpty**
 - check if stack is empty.

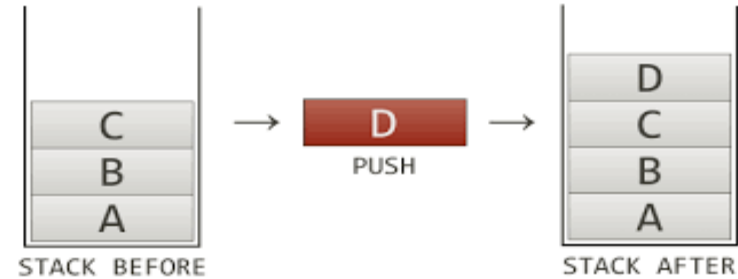


Stack Operation Example



Stack Implementations

- Array based
 - Where is top?
 - How are elements added, removed?



- Linked List based
 - Where is top?
 - How are elements added, removed?



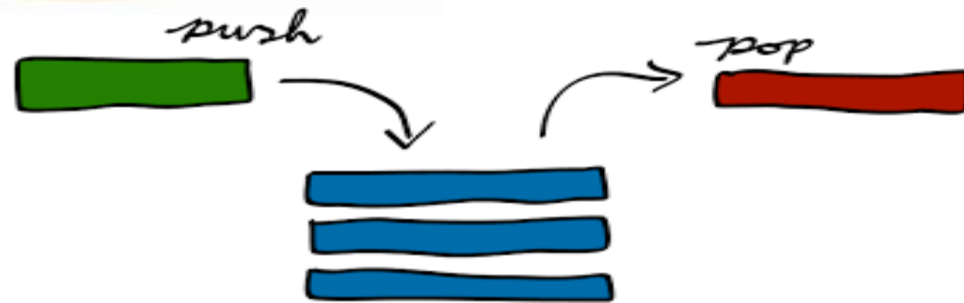
APPLICATIONS OF STACK

- Page-visited history in a Web browser
- Undo sequence in a text editor
- Saving local variables when one function calls another, and this one calls another, and so on. (Recursion)
- Expression evaluation and syntax parsing
- Backtracking



Some uses of stacks

- Stacks are used for:
 - Any sort of nesting (such as parentheses)
 - Evaluating arithmetic expressions (and other sorts of expression)
 - Implementing function or method calls
 - Keeping track of previous choices (as in backtracking)
 - Keeping track of choices yet to be made (as in creating a maze)



The Queue ADT

- With a queue, insertion is done at one end whereas deletion is performed at the other end.
- It is a **First In First Out (FIFO)** Structure. That is, the first element to enter into the queue will be the first element to get out of the queue.

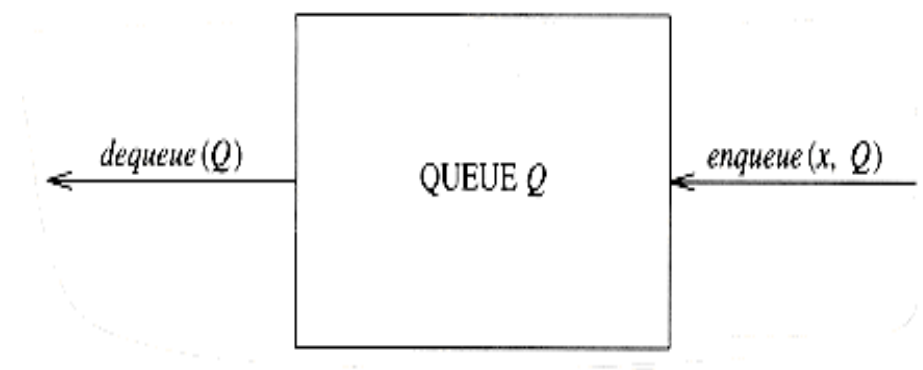
Front



Rear

The basic operations on a Queue

- Enqueue: inserts an element at the end of the list (called the rear)
- Dequeue: deletes (and returns) the element at the start of the list (called the front).

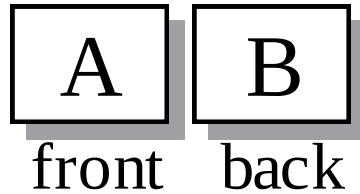


- Peek: Get the front element

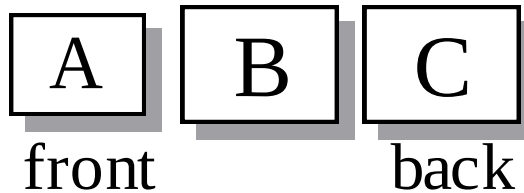
OPERATIONS OF A QUEUE



push A **PUSH** (ALSO CALLED **ENQUEUE**) -- TO
INSERT AN ITEM AT THE BACK

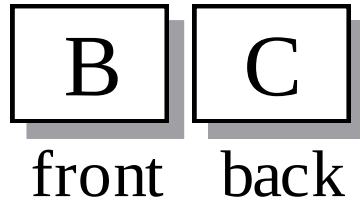


push B **POP** (ALSO CALLED **DEQUEUE**) -- TO
DELETE THE FRONT ITEM



push C

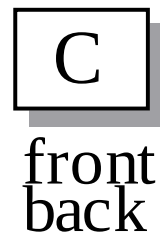
IN A QUEUE, THE FIRST ITEM



pop A

INSERTED WILL BE THE FIRST ITEM

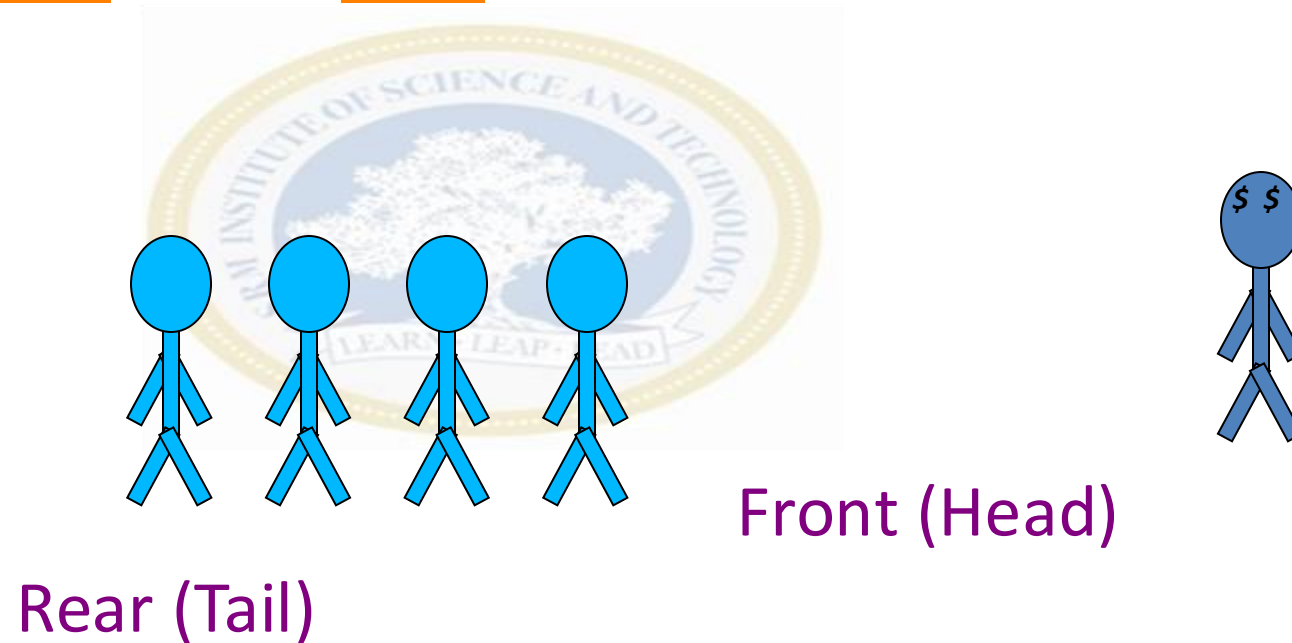
DELETED: FIFO (FIRST-IN, FIRST-OUT)



pop B

The Queue Operations

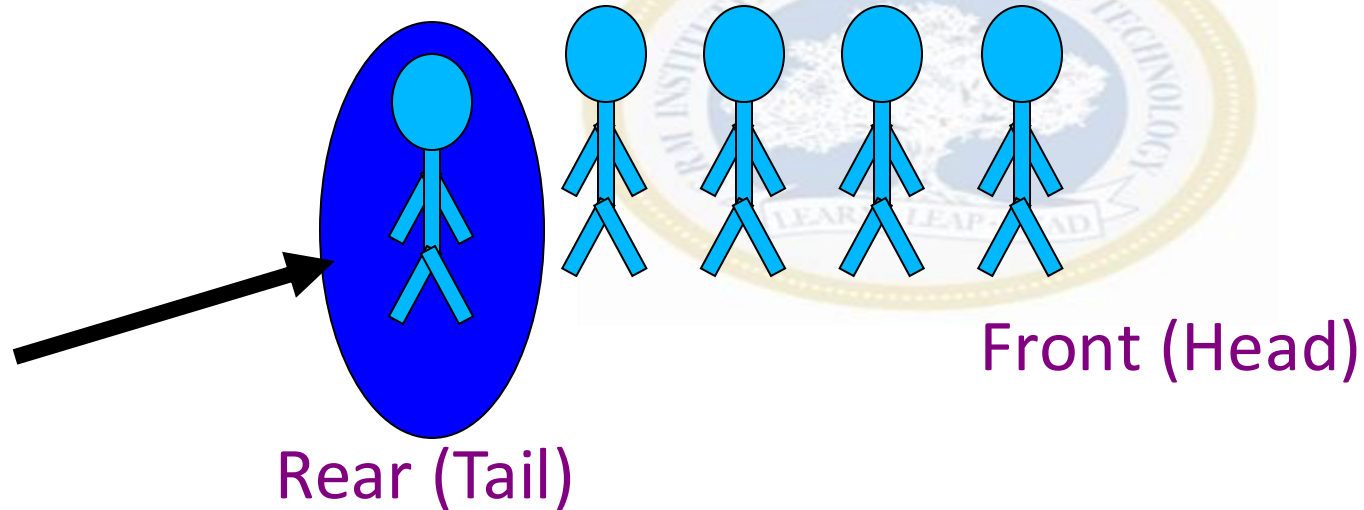
- A queue is like a line of people waiting for a bank teller. The queue has a front and a rear.





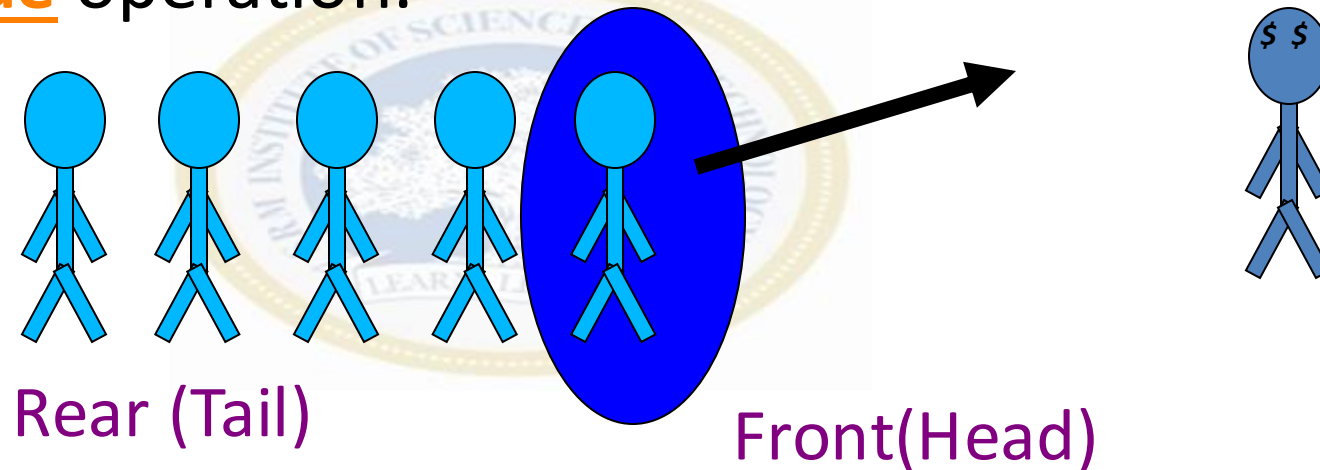
The Queue Operations

- New people must enter the queue at the rear. The C++ queue class calls this a insert, although it is usually called an enqueue operation.



The Queue Operations

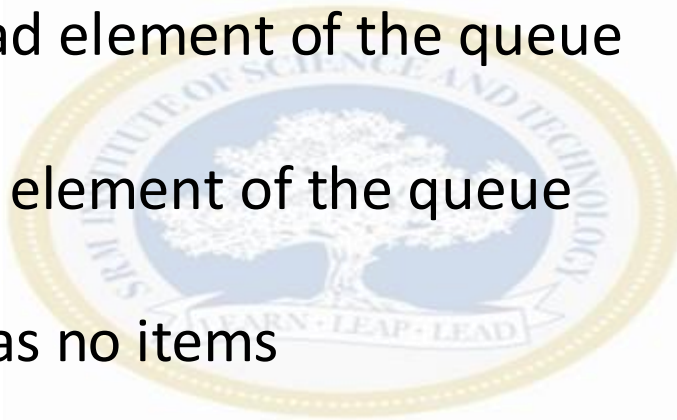
- When an item is taken from the queue, it always comes from the front. The C++ queue calls this a delete, although it is usually called a dequeue operation.





Operations on Queues

- **Insert(item):** (also called enqueue)
 - It adds a new item to the tail of the queue
- **Remove():** (also called delete or dequeue)
 - It deletes the head item of the queue, and returns to the caller. If the queue is already empty, this operation returns NULL
- **getHead():**
 - Returns the value in the head element of the queue
- **getTail():**
 - Returns the value in the tail element of the queue
- **isEmpty()**
 - Returns **true** if the queue has no items
- **size()**
 - Returns the number of items in the queue



Queue Representations

- Queues can be represented using:
 - Arrays
 - Linked Lists



Applications of Queue

- Operating systems often maintain a queue of processes that are ready to execute or that are waiting for a particular event to occur.
- Computer systems must often provide a “holding area” for messages between two processes, two programs, or even two systems. This **holding area is usually called a “buffer” and is often implemented as a queue.**
- Serving requests on a single shared resource, like a printer, CPU task scheduling etc.
- In real life, Call Center phone systems will use Queues, to hold people calling them in an order, until a service representative is free.
- Handling of interrupts in real-time systems. The interrupts are handled in the same order as they arrive, First come first served.



PROS AND CONS OF QUEUES

- Easy to implement
- But it has a limited capacity with a fixed array
- Or you must use a dynamic array for an unbounded capacity
- Special behaviour is needed when the rear reaches the end of the array.
(**Overcome by CIRCULAR QUEUES**)

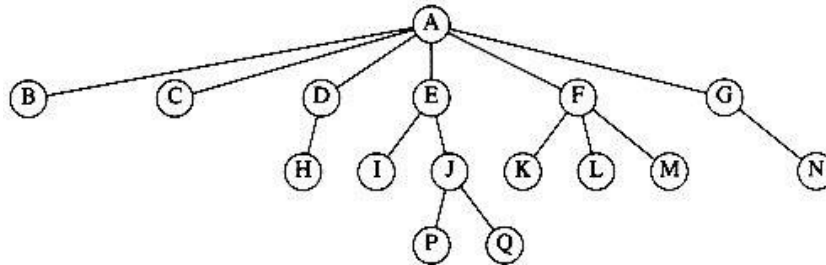
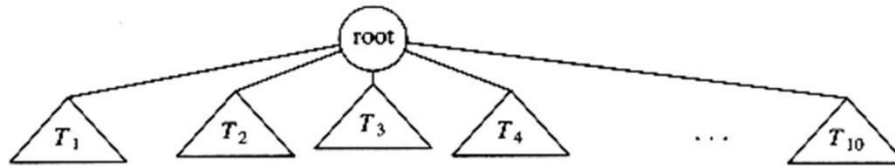


ADVANTAGES OF CIRCULAR QUEUES

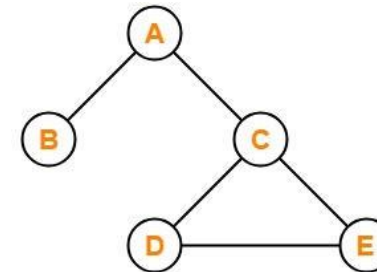
- A standard queue suffers from a rebuffering problem during dequeue operations.
- By making the queue circular and linking the head to the tail, this alleviates the problem and allows insertion and deletion in constant time.

General Trees

- Tree consists of collection of nodes arranged in hierarchical pattern



Tree - Examples



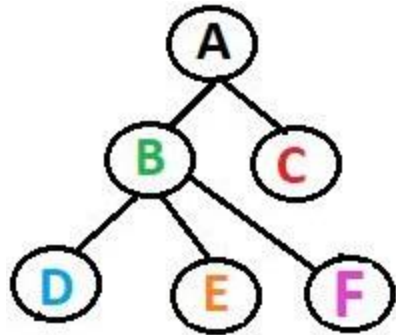
Not a tree

Tree Terminology

- ☐ Root
- ☐ Edge
- ☐ Parent
- ☐ Child
- ☐ Sibling
- ☐ Degree
- ☐ Internal node
- ☐ Leaf node
- ☐ Level
- ☐ Height
- ☐ Depth
- ☐ Subtree

Tree Terminology - Degree

- Degree of node – number of children per node
- Degree of tree – highest degree of a node among all nodes in tree



Degree A – 2

Degree B – 3

Degree C – 0

Degree D – 0

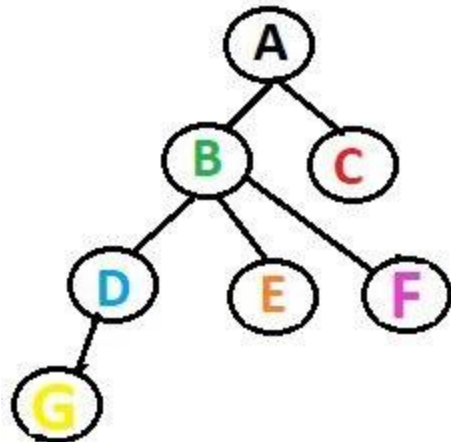
Degree E – 0

Degree F – 0

Degree of entire tree - 3

Tree Terminology – Height

- Number of edges in longest path from the node to any leaf
- Height of any leaf node is 0
- Height of Tree = Height of Root node



Height A – 3

Height B – 2

Height D – 1

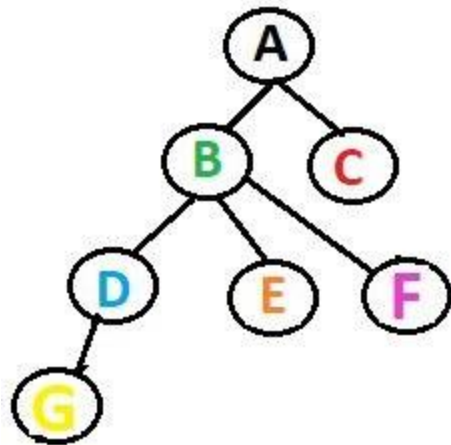
Height C,G,E,F – 0

Height of tree - 3



Tree Terminology – Depth

- Number of edges from root node to particular node is Depth
- Depth of root node is 0
- Depth of Tree = Depth of longest path from root to leaf



Depth A – 0

Depth B ,C – 1

Depth D, E, F – 2

Depth G – 3

Depth of tree - 3



Uses of trees

1. Manipulate hierarchical data.
2. Make information easy to search.
3. Manipulate sorted lists of data.
4. Router algorithms.

Applications of trees

- **Represent organization**

The library database in a library, a student database in a school or college, an employee database in a company, a patient database in a hospital or any database for that matter would be implemented using trees.

- **Represent computer file systems**

The file system in your computer i.e. folders and all files, would be stored as a tree.

When you search for a word in a file or misspell a word and you get a list of possible correct words, you are using a tree

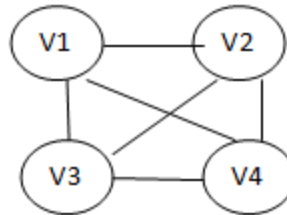
- **Networks to find best path in the Internet**

When you watch a YouTube video or surf anything on the internet, the data which would be present in a computer somewhere in the world would travel to your computer passing through many intermediate computers called routers. The routers heavily use trees for routing.

- **Google maps uses trees for all the locations in it.**

GRAPH DEFINITION

- A graph $G = (V, E)$ consists of a set of vertices V and set of edges E .
- Vertices are often also called nodes.
- Elements of E are called edges, or directed edges, or arcs.
- Each edge is a pair (v, w) where $v, w \in V$

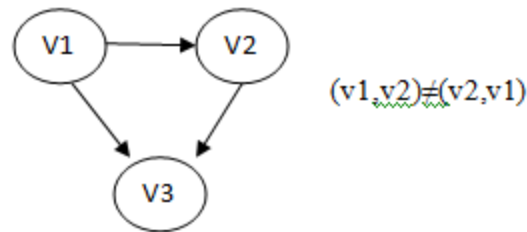


Here v_1, v_2, v_3, v_4 are the vertices and $(v_1, v_2), (v_2, v_3), (v_3, v_4), (v_4, v_1), (v_2, v_4), (v_1, v_3)$ are edges.

Basic Terminologies

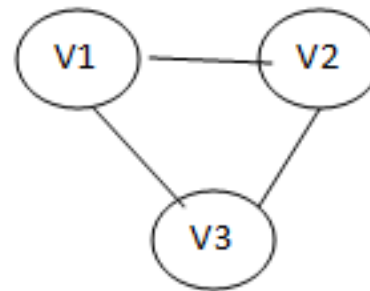
Directed Graph

- A directed graph, or digraph, is a graph which consists of directed edges where each edge in E is unidirectional.
- If (v,w) is a directed edge then $(v,w) \neq (w,v)$
- For directed edge (v, w) in E , v is its tail and w its head;
- (v, w) is represented in the diagrams as the arrow, $v \rightarrow w$.



Undirected Graph

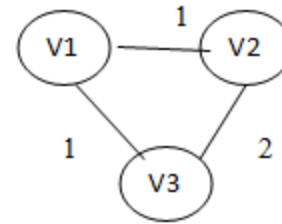
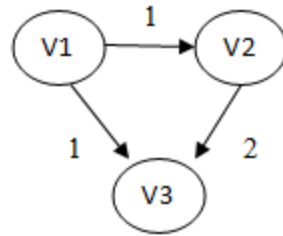
- A undirected graph is a graph which consists of undirected edges. If (v,w) is an undirected edge then $(v,w) = (w,v)$



$$(v1, v2) = (v2, v1)$$

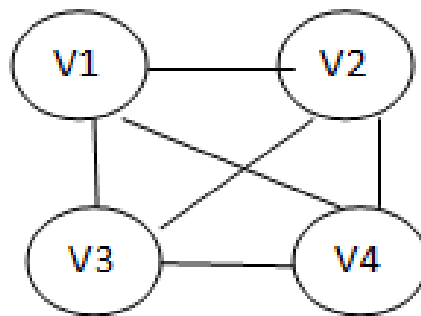
Weighted Graph

- A graph is said to be weighted graph if every edge in a graph is assigned a weight or value. It can be a directed or undirected graph



Complete Graph

- A complete graph is a graph in which there is an edge between every pair of vertices. A complete graph with n vertices will have $n(n-1)/2$ edges.



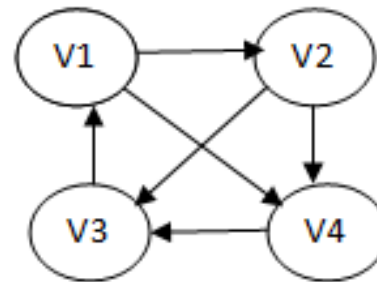
$$n=4$$

$$(4(4-1)/2)=4*3/2=6 \text{ edges}$$

No of vertices=4, No of edges=6

Degree

- The **number of edges incident on a vertex determines its degree**. The degree of the vertex is written as $\text{degree}(v)$. The in degree of the vertex V is the number of edges entering into vertex V . Similarly the out degree of the vertex V is the number of edges existing from that vertex V .

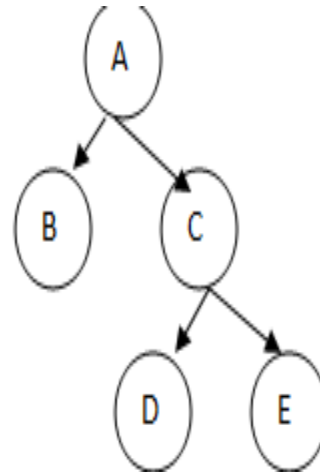


Outdegree $(v1)=2$

Indegree $(v1)=1$

Acyclic Graph

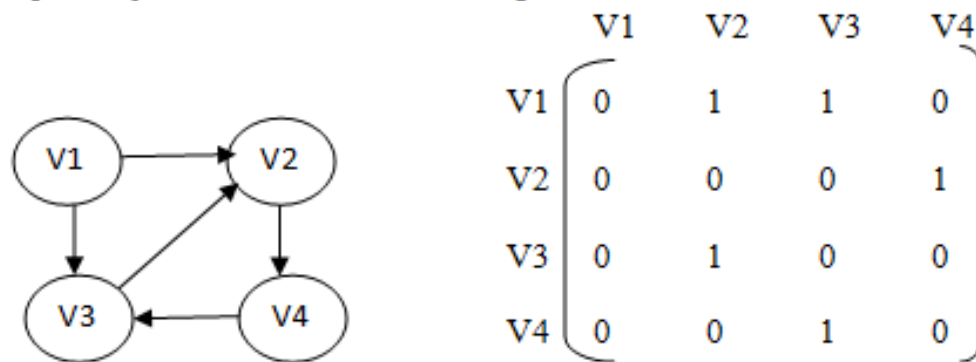
- A directed graph which has no cycles is referred to as acyclic graph. It is abbreviated as DAG (Directed Acyclic Graph)



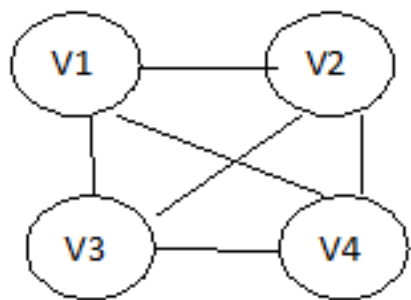
Representation of Graph

- Graph can be represented by Adjacency Matrix and Adjacency List.
- **Adjacency Matrix Representation**
- The adjacency matrix A for a graph $G=(V,E)$ with n vertices in an $n \times n$ matrix, such that
- $A_{ij} = 1$, if there is an edge V_i to V_j
- $A_{ij} = 0$, if there is no edge

(a) Adjacency Matrix for Directed Graph



(a) Adjacency Matrix for Undirected Graph



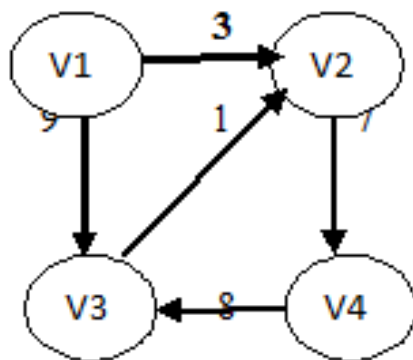
	V1	V2	V3	V4
V1	0	1	1	0
V2	1	0	1	1
V3	1	1	0	1
V4	1	1	1	0

(a) Adjacency Matrix for Weighted Graph

$A_{ij} = C_{ij}$, if there is an edge from V_i to V_j

$A_{ij} = 0$, if there is no edge and $i \neq j$.

If there is no arc from i to j , assume $c[i,j] = \infty$ where $i \neq j$



	V1	V2	V3	V4
V1	0	3	9	
V2		0		7
V3		1	0	
V4		1	8	0

Pros:

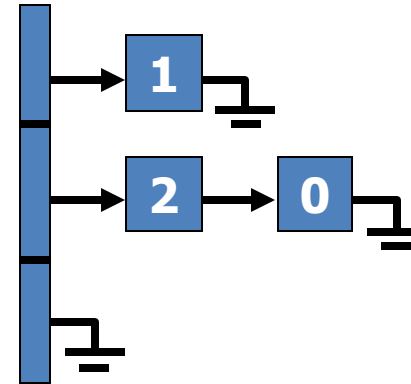
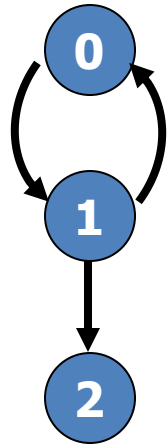
- Simple to implement
- Easy and fast to tell if a pair (i,j) is an edge: simply check if $A[i][j]$ is 1 or 0

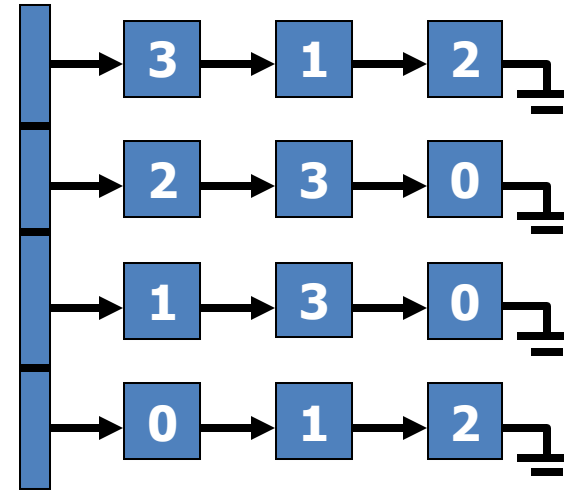
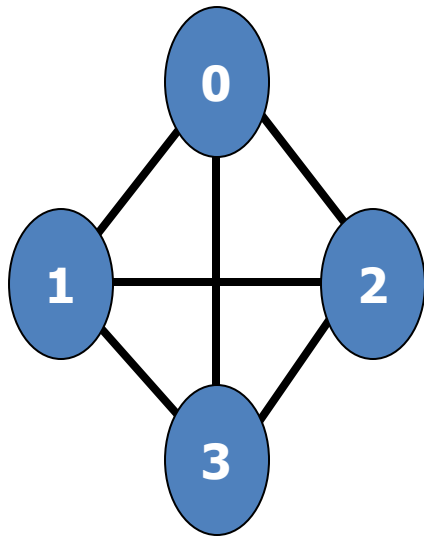
Cons:

- No matter how few edges the graph has, the matrix takes $O(n^2)$ in memory

Adjacency List Representation

- In this representation we store a graph as a linked structure. We store all the vertices in a list for each vertex; we have a linked list of its adjacency vertices.





Pros:

- Saves on space (memory): the representation takes as many memory words as there are nodes and edge.

Cons:

- It can take up to $O(n)$ time to determine if a pair of nodes (i,j) is an edge: one would have to search the linked list $L[i]$, which takes time proportional to the length of $L[i]$.

Search

- Searching is the universal technique of problem solving in Artificial Intelligence
- A search problem consists of:
 - **A State Space.** Set of all possible states where you can be.
 - **A Start State.** The state from where the search begins.
 - **A Goal Test.** A function that looks at the current state returns whether or not it is the goal state.
 - The **Solution** to a search problem is a sequence of actions, called the **plan** that transforms the start state to the goal state.

Search Problem Components

- **Search tree**: A tree representation of search problem is called Search tree. The root of the search tree is the root node which is corresponding to the initial state.
- **Actions**: It gives the description of all the available actions to the agent.
- **Transition model**: A description of what each action do, can be represented as a transition model.
- **Path Cost**: It is a function which assigns a numeric cost to each path.
- **Solution**: It is an action sequence which leads from the start node to the goal node.
- **Optimal Solution**: If a solution has the lowest cost among all solutions.

General Search Algorithm

For any problem P , we have a set of initial states, say I , and a set of goal states, say G . We start with the first state and proceed till either we get a solution or return to failure.

Let us assume that the list L comprises the nodes or the states from I . The algorithm is described as follows:

While list L is not empty or has a solution

{

 Remove the state X from L .

 Take the next decision for the successor.

 Generate a new state for each possible choice for the decision.

 Merge/re-order the new states with L .

}

If L not empty (has a solution), return success.

Else return failure.

Examples of Search Algorithm

- TSP
- 8 Puzzle problem
- Tic Tac Toe
- N-queen problem
- Tower of Hanoi
- Water Jug Problem
- Blocks World
- Vacuum Cleaner

Vacuum Cleaner

Example: Vacuum-Cleaner

States

- 8 states

Initial state

- any state

Actions

- Left, Right, and Suck

Transition model

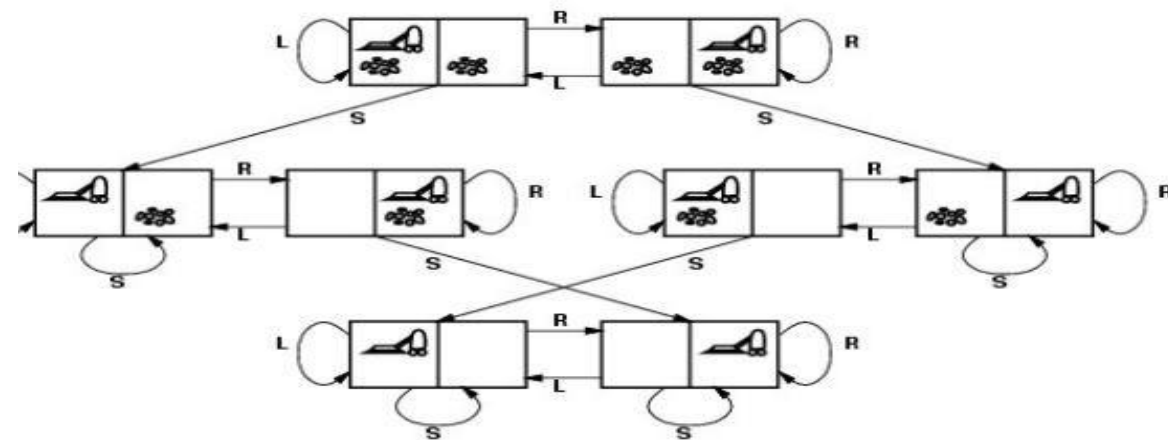
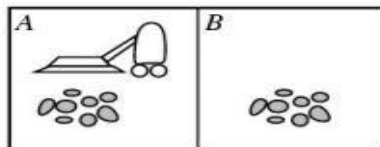
- complete state space, see next page

Goal test

- whether both squares are clean

Path cost

- each step costs 1



8 Puzzle Problem

Example: 8-puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

NP-Complete

- States:
 - location of each tile and the blank
- Initial state: any, $9!/2$
- Actions:
 - blank moves Left, Right, Up or Down
- Transition model:
 - Given a state and action, returns the resulting state
- Goal test: Goal configuration
- Path cost: Each step costs 1

Agents

- An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators
- Human agent:
 - eyes, ears, and other organs for sensors;
 - hands, legs, mouth, and other body parts for actuators
- Robotic agent:
 - cameras and infrared range finders for sensors
 - various motors for actuators

Rational agents

- An agent is an entity that perceives and acts
- Abstractly, an agent is a function from percept histories to actions:

$$[f: P^* \rightarrow A]$$

- For any given class of environments and tasks, we seek the agent (or class of agents) with the best performance
- Computational limitations make perfect rationality unachievable
→ design best program for given machine resources

Problem-solving Agents

The following steps describe the process:

1. Formulate goal.
2. Formulate the problem which has the goal and the initial states → problem.
3. Now search with the given problem → sequence of actions.
4. Action on the sequence → Act according to the sequence of actions.

Let us proceed towards how control strategies help and play a role in the search techniques.

Control Strategies

- Search is a general algorithm that helps in finding the path in state space
- The path may lead to the solution or dead end.
- Control strategies- overall rules and approach towards searching
 - i) forward search(data directed)

Starts search from initial state towards goal state.
Ex: locating a city from current location
 - ii) backward search(goal directed)

Search starts from goal state towards a solvable initial state.
Ex: start from target city

Control Strategies

- Strategies to explore the states
 - Informed search - **No guarantee for solution** but high probability of getting solution
 - **heuristic approach** is used to control the flow of solution path
 - heuristic approach is a technique based on common sense, rule of thumb, educated guesses or intuitive judgment
 - Uninformed search - **generates all possible states in the state space** and checks for the goal state.
 - **time consuming** due to large state space
 - used where error in the algorithm has severe consequences

Control Strategies

Systematic search: A systematic strategy is used when the search space is small and systematic, but blind. We call it blind because this is the case where the search has no information about its domain. The only thing it can do is just distinguishing between the goal and non-goal states. For example, you are to locate a particular book in a library, where the books are not indexed and arranged properly. Then, there would be no option but to go blindly and check every book rack. So, systematically, you try to cover all book racks.

Depth First search and Breadth First Search

Control Strategies

Heuristic search: *Heuristic* is typically a technique based on the previous experience and provides guidelines to solve the problems. It is a technique that improves the efficiency of a search process. It is exploratory in nature. Imagine a teacher assisting a student in developing some layout of a document. In this process, the teacher may give some important hints to help the student in doing it in the best possible way or may miss some of them. The function of the teacher can be said to have a heuristic approach. Remember that heuristic search may not find out the best solution, but it guarantees to find a good solution in a reasonable amount of time. It uses some measure of relative merits, which can guide the search. For example, if you are locating a city, air distance of the city from current location can guide you to select the direction. This approach is most commonly used where the systems have time or space constraints, and where the dependency is on the knowledge of the problem domain. So, they rely on the promising lines of development to proceed.

Parameters for Search Evaluation

- Parameters for search evaluation
 - i) completeness: Guaranteed to find a solution within finite time
 - ii) space and time complexity: memory required and time factor needed
 - iii) optimality and admissibility: correctness of the solution

Uninformed and Heuristic Search Strategies

- Based on the information about the problem available for the search strategies, the search algorithms are classified into uninformed (Blind) and informed (or heuristic) search algorithms.
- For the **uninformed search algorithms**, the strategies have no additional information about the states beyond that provided by the problem definition. Generate successors and differentiate between goal and non-goal states.
- Strategies that know whether one non-goal state is more promising than the other is called **informed search strategies** or **heuristic search strategies**.

Uninformed Search

- The uninformed search does not contain any domain knowledge such as closeness, the location of the goal.
- It operates in a **brute-force** way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes.
- Uninformed search applies a way in which search tree is searched without any information about the search space like initial state operators and test for the goal, so it is also called **blind search**.
- It examines each node of the tree until it achieves the goal node.

Uninformed Search methods

Uninformed search strategies use only the information available in the problem definition

Breadth-first search

Uniform-cost search

Depth-first search

Depth-limited search

Iterative deepening search

BFS

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first search implemented using FIFO queue data structure.

Breadth First Search(BFS)

- **Breadth First Search (BFS)** is an algorithm for traversing an unweighted Graph or a Tree.
- BFS starts with the root node and explores each adjacent node before exploring node(s) at the next level.
- BFS makes use of **Queue** for storing the visited nodes of the graph / tree.

Steps

- Choose any node in the graph, designate it as source node and mark it as visited.
- Using the adjacency matrix of the graph find all the unvisited adjacent nodes to the search node and enqueue them into the queue Q
- Then the node is dequeued from the queue. Mark that node as visited and designate it as the new search node.
- Repeat step 2 and 3 using the new search node.
- This process continues until the queue Q which keeps track of the adjacent nodes is empty.

Breadth-First Search Algorithm:

Algorithm : Breadth first search (Graph G, Source_Vertex S)

1. Create a queue **Q** to store the vertices.
2. Push the source vertex **S** in the queue **Q**.
3. Mark **S** as visited.
4. **While** the queue **Q** is not empty
5. Remove vertex **U** from the front of the queue. i.e Vertex **U** = **Q**.front(), **Q**.pop()
6. **For** every vertex **V** adjacent to the vertex **U**
7. **If** the vertex **V** is not visited **Then**
8. Explore the vertex **V** and mark **V** as visited.
9. Push the vertex **V** in the queue **Q**.

BFS Algorithm

function BREADTH-FIRST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

frontier $\leftarrow$ a FIFO queue with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the shallowest node in *frontier* */

 add *node*.STATE to *explored*

for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

if *child*.STATE is not in *explored* or *frontier* **then**

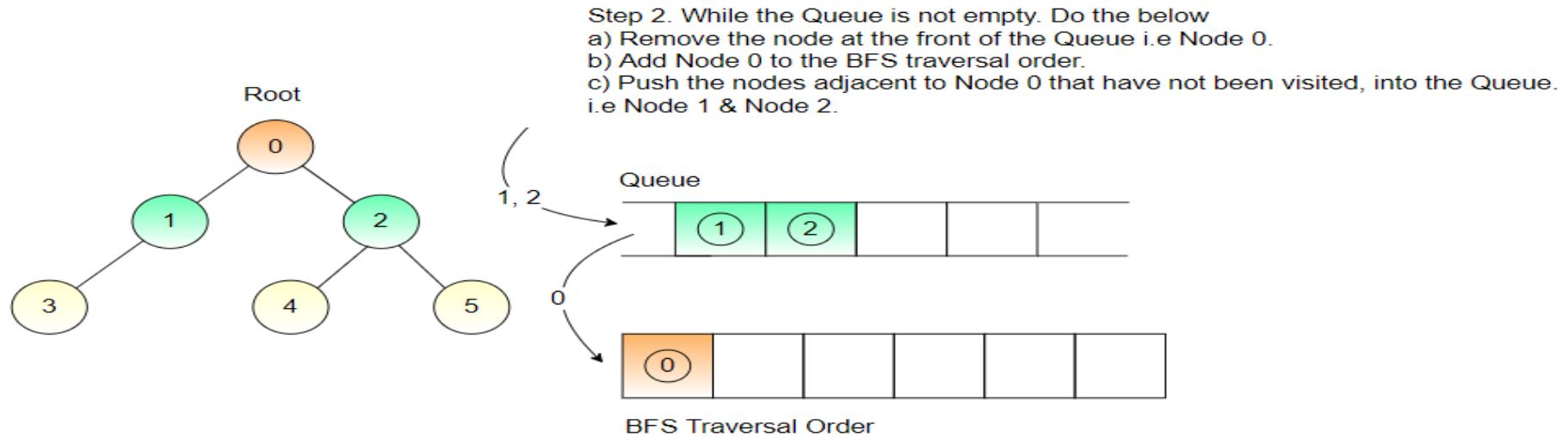
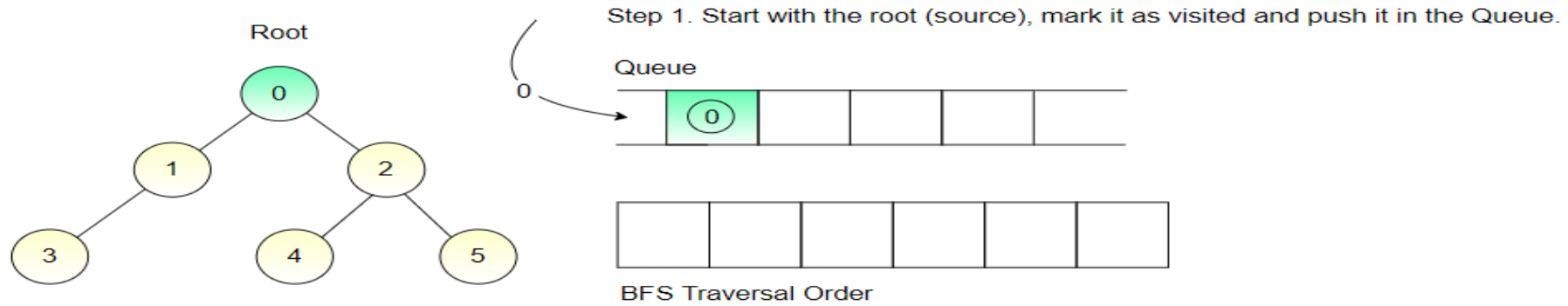
if *problem*.GOAL-TEST(*child*.STATE) **then return** SOLUTION(*child*)

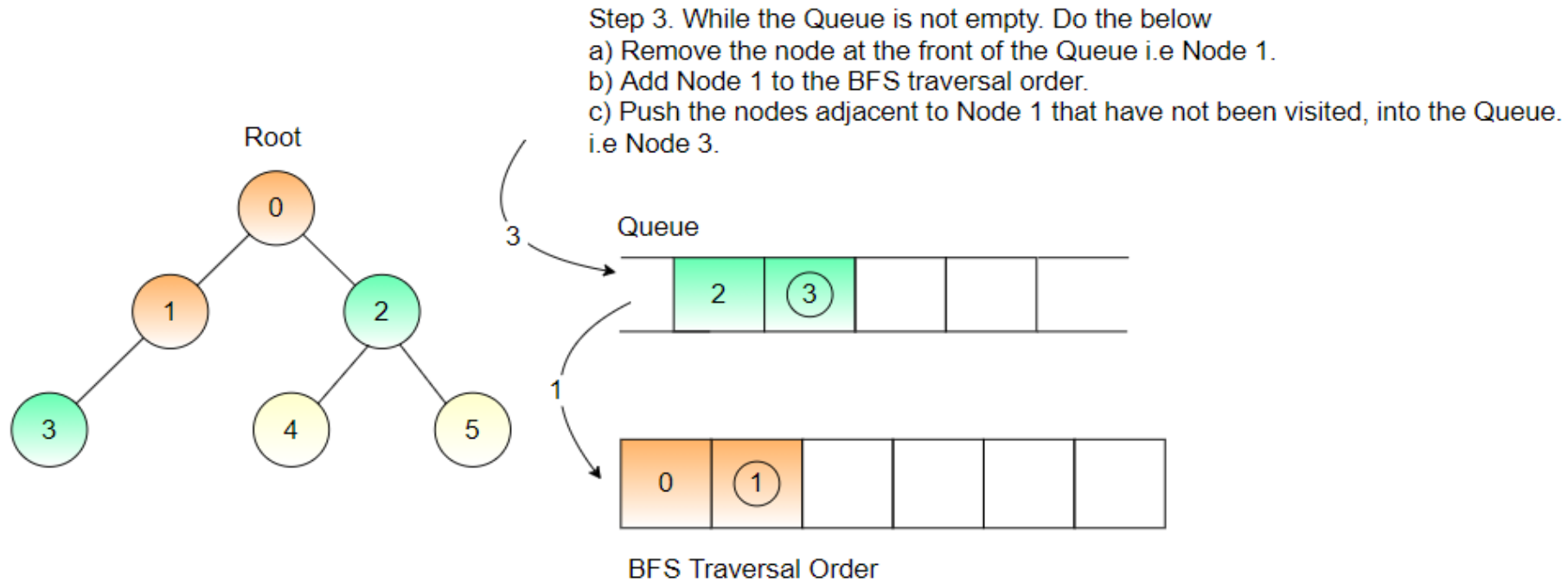
frontier $\leftarrow$ INSERT(*child*, *frontier*)

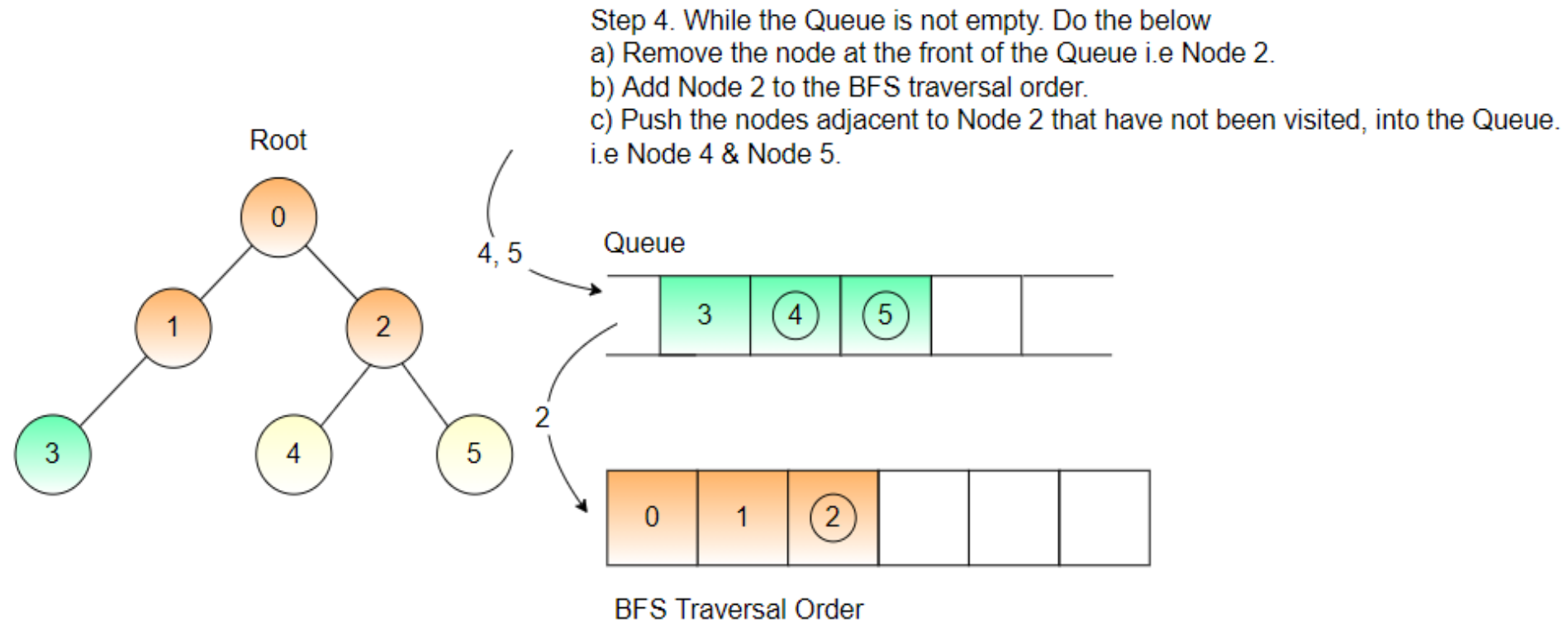
Child Node

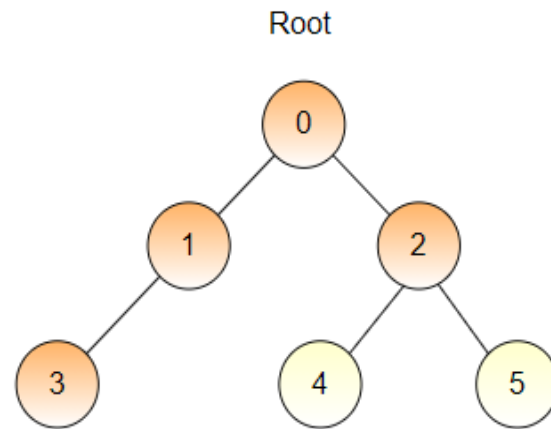
```
function CHILD-NODE(problem, parent, action) returns a node
    return a node with
        STATE = problem.RESULT(parent.STATE, action)
        PARENT = parent
        ACTION = action
        PATH-COST = parent.PATH-COST +
                    problem.STEP-COST(parent.STATE, action)
```

Example-1









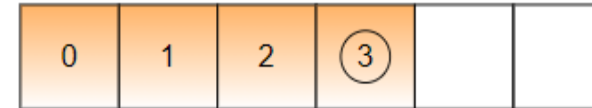
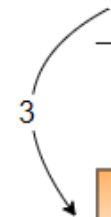
Step 5. While the Queue is not empty. Do the below

a) Remove the node at the front of the Queue i.e Node 3.

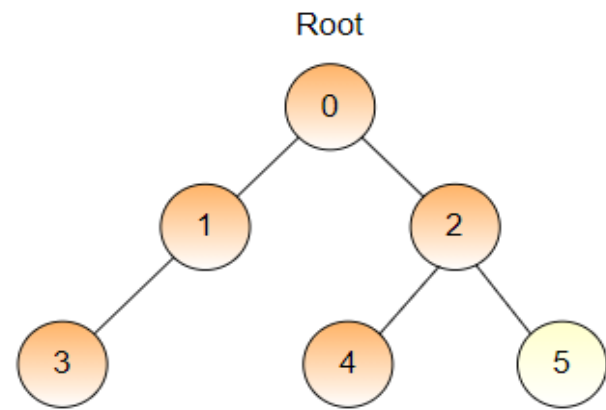
b) Add Node 3 to the BFS traversal order.

c) Push the nodes adjacent to Node 3 that have not been visited, into the Queue.
i.e **None**, as the only node adjacent to Node 3 is Node 1 that has already been visited.

Queue



BFS Traversal Order



Step 6. While the Queue is not empty. Do the below

a) Remove the node at the front of the Queue i.e Node 4.

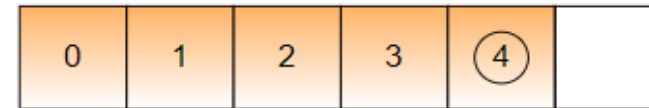
b) Add Node 4 to the BFS traversal order.

c) Push the nodes adjacent to Node 4 that have not been visited, into the Queue.
i.e **None**, as the only node adjacent to Node 4 is Node 2 that has already been visited.

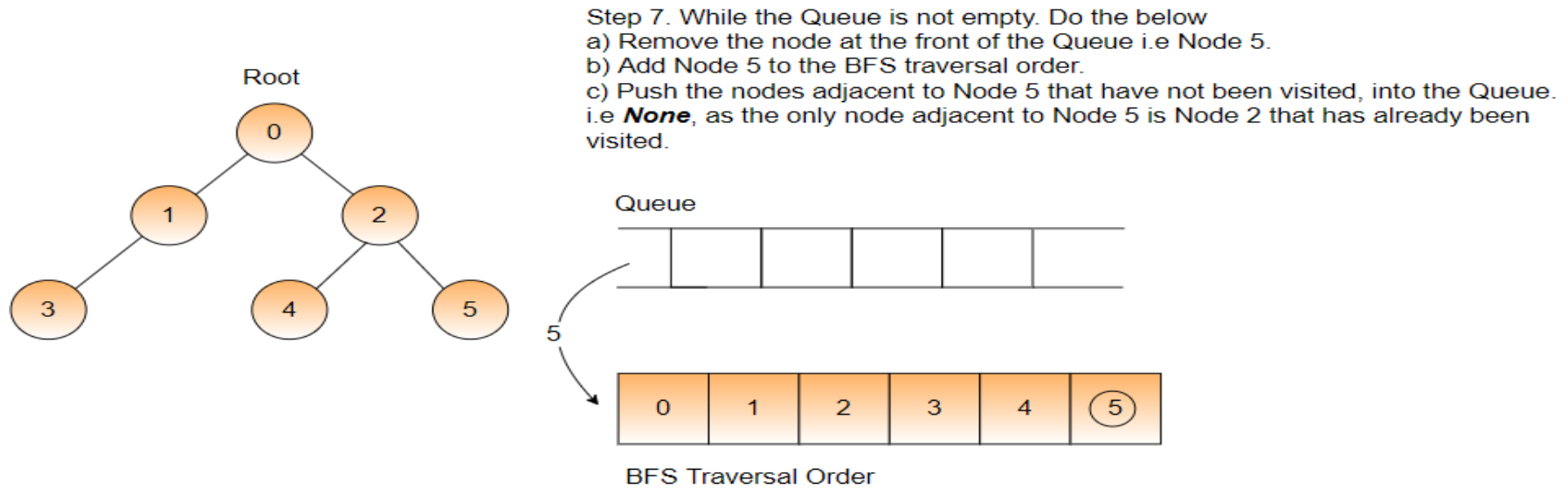
Queue



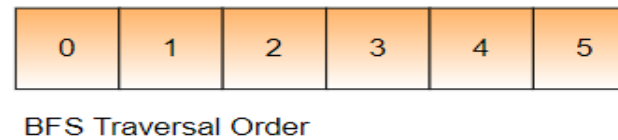
4



BFS Traversal Order



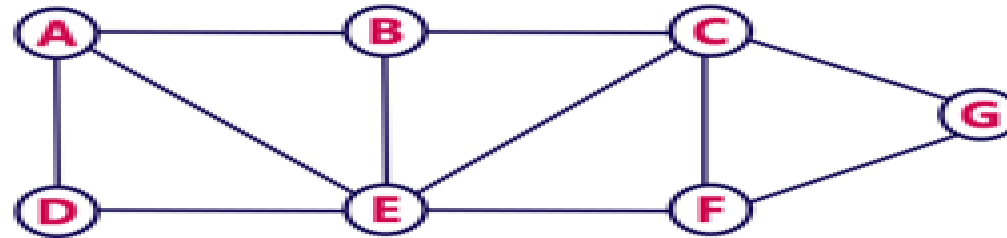
The Queue is now empty, thus all the nodes are visited in the BFS order.



EXAMPLE-2

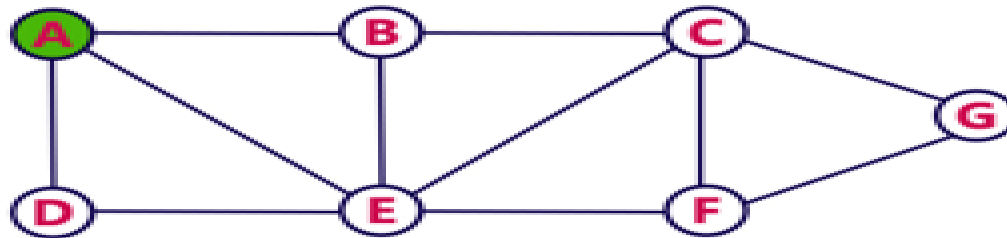
Breadth-First Search Algorithm:

Consider the following example graph to perform BFS traversal



Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Insert **A** into the Queue.



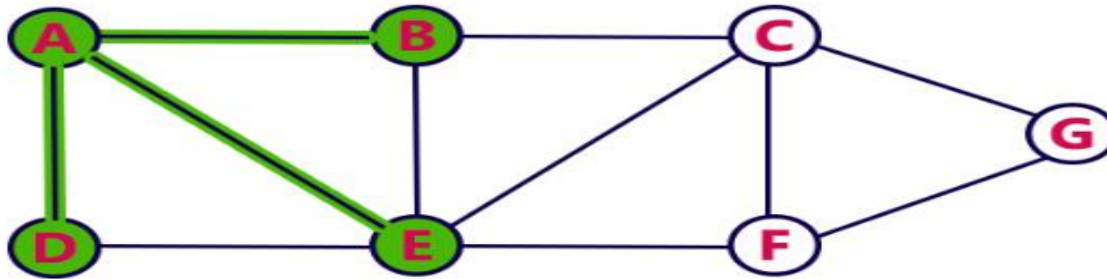
Queue



Breadth-First Search Algorithm:

Step 2:

- Visit all adjacent vertices of **A** which are not visited (**D, E, B**).
- Insert newly visited vertices into the Queue and delete A from the Queue..

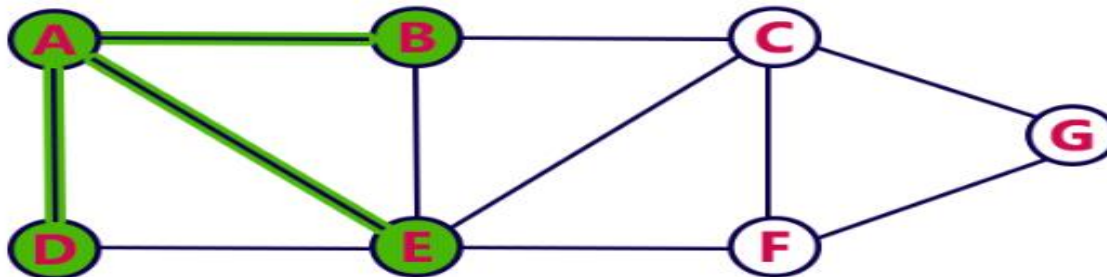


Queue



Step 3:

- Visit all adjacent vertices of **D** which are not visited (there is no vertex).
- Delete D from the Queue.



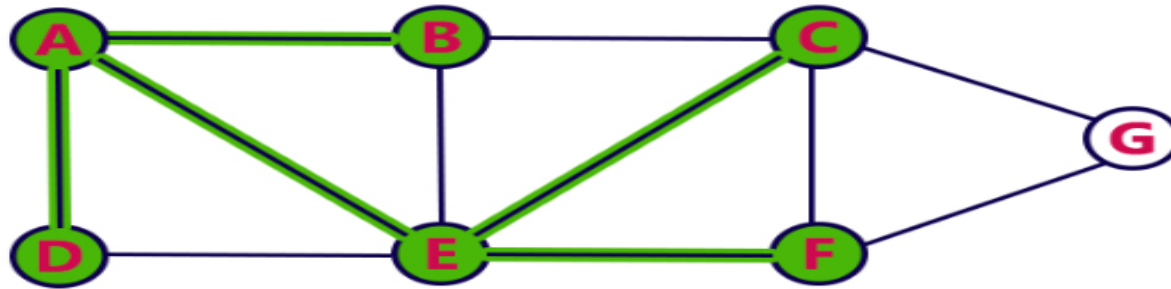
Queue



Breadth-First Search Algorithm:

Step 4:

- Visit all adjacent vertices of **E** which are not visited (**C**, **F**).
- Insert newly visited vertices into the Queue and delete E from the Queue.

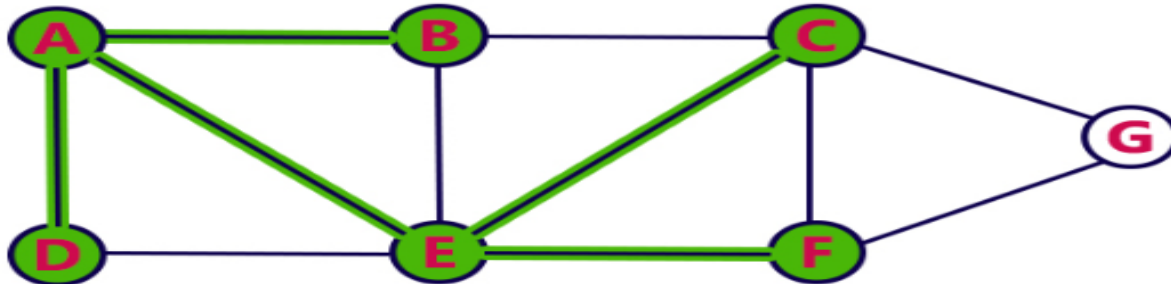


Queue



Step 5:

- Visit all adjacent vertices of **B** which are not visited (**there is no vertex**).
- Delete **B** from the Queue.



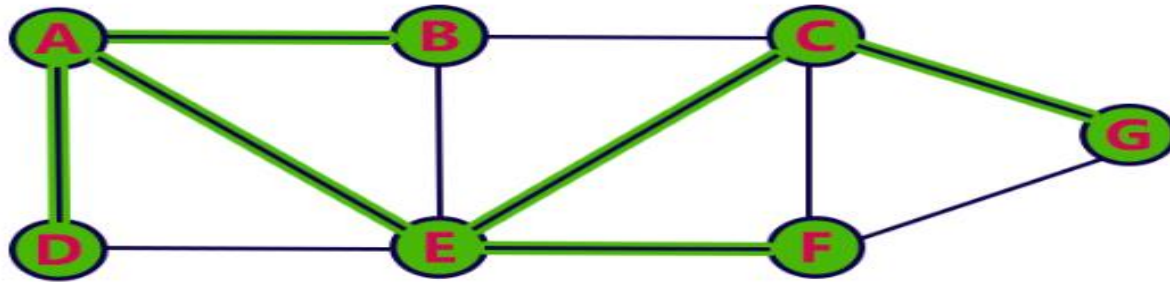
Queue



Breadth-First Search Algorithm:

Step 6:

- Visit all adjacent vertices of **C** which are not visited (**G**).
- Insert newly visited vertex into the Queue and delete **C** from the Queue.

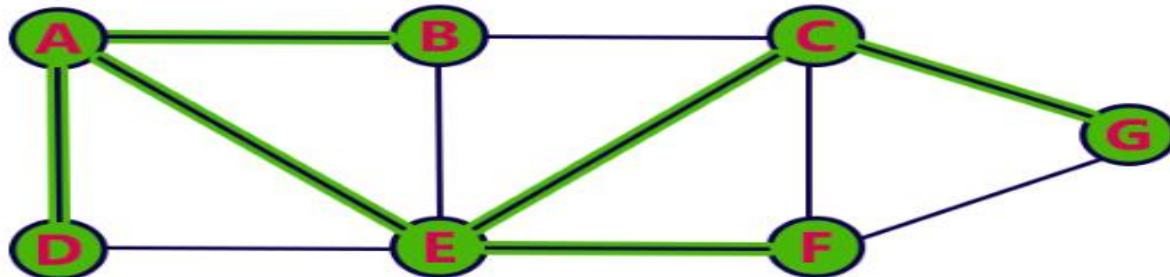


Queue



Step 7:

- Visit all adjacent vertices of **F** which are not visited (**there is no vertex**).
- Delete **F** from the Queue.



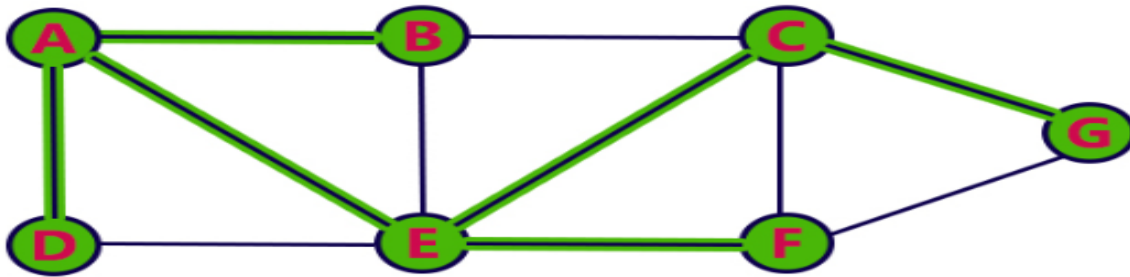
Queue



Breadth-First Search Algorithm:

Step 8:

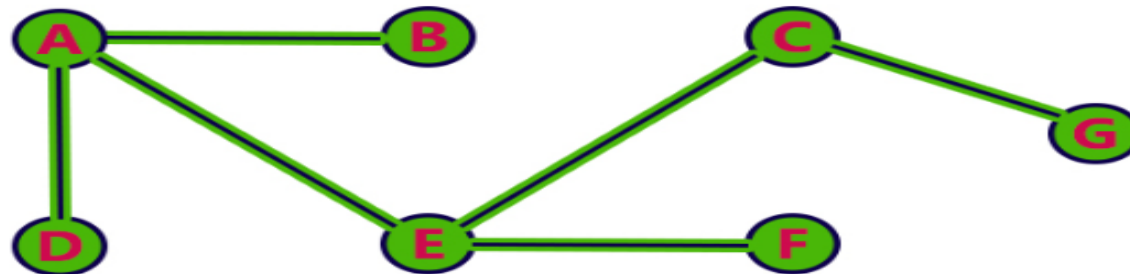
- Visit all adjacent vertices of **G** which are not visited (**there is no vertex**).
- Delete **G** from the Queue.



Queue



-
- Queue became Empty. So, stop the BFS process.
 - Final result of BFS is a Spanning Tree as shown below...



Features of Breadth-First Search Algorithm:

- *Space complexity*

- In the breadth-first search algorithm, all the nodes at a particular level must be saved until their child nodes in the next level have been generated.
- The space complexity is therefore proportional to the number of nodes at the deepest level of the graph. Given a graph with branching factor b (number of children at each node) and depth d , the asymptotic space complexity is the number of nodes at the deepest level $O(bd)$.

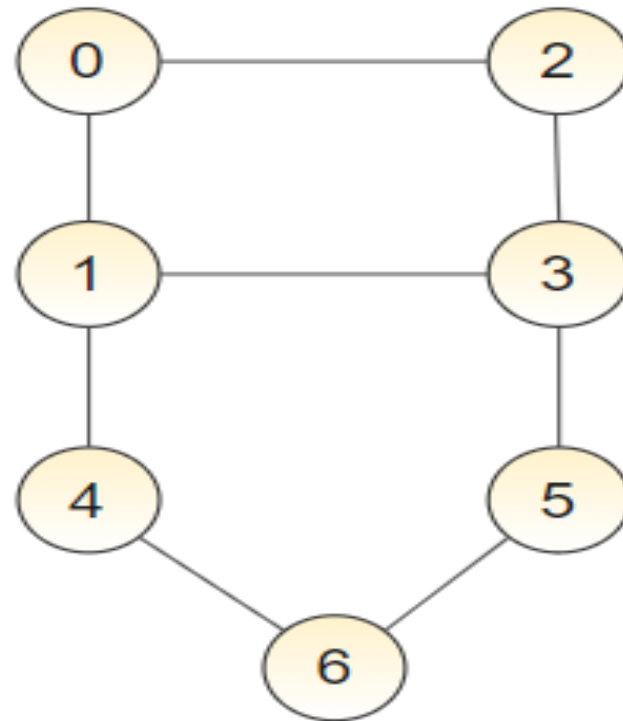
- *Time complexity*

- In the worst case, breadth-first search has to traverse through all paths to all possible nodes, thus the time complexity of this algorithm asymptotically approaches $O(bd)$.
- However, the time complexity can also be expressed as $O(|E| + |V|)$, since every vertex and every edge will be explored in the worst case.

Application of BFS

- **GPS Navigation systems**
- **Computer Networks**
- **Face book**

Find the solution:



Depth First Search (DFS)

- DFS is an algorithm for traversing a Graph or a Tree.
- DFS starts with the root node and explores all the nodes along the depth of the selected path before backtracking to explore the next path.
- DFS makes use of Stack for storing the visited nodes of the graph / tree.

Steps

- Choose any node in the graph. Designate it as the search node and mark it as visited.
- Using the adjacency matrix of the graph, find a node adjacent to the search node that has not be visited yet. Designate this as the new search node and mark it as visited.
- Repeat step 2 using the new search node. If no nodes satisfying (2) can be found return to the previous search node and continue from there.
- When a return to the previous search node in (3) is impossible, the search from the originally chosen search node is complete.
- If the graph still contains unvisited nodes, choose any node that has not been visited and repeat step 1 through 4.

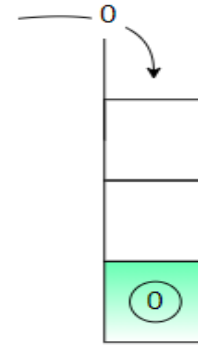
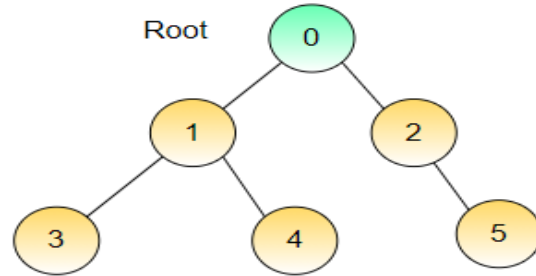
Depth-first Search Algorithm:

Algorithm : Depth first search (Graph G, Souce_Verterx S)

1. Create a stack **STK** to store the vertices.
 2. Push the source vertex **S** in the stack 'STK'.
 3. **While** the stack **STK** is not empty
 4. Pop the vertex **U** from the top of the stack. i.e Vertex **U** = **STK.top()**, **STK.pop()**
 5. **If** the vertex **U** is not visited, **Then**
 6. Explore the vertex **U** and mark **U** as visited.
 7. **For** every vertex **V** adjacent to vertex **U**
 8. Push the vertex **V** in the stack **STK**
-

Example-1

Step 1. Start with the root (source), mark it as visited and push it in the Stack.



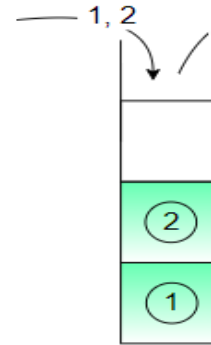
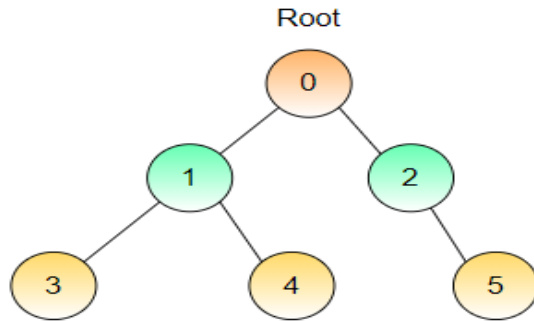
DFS Traversal Order

Step 2. While the Stack is not empty. Do the below

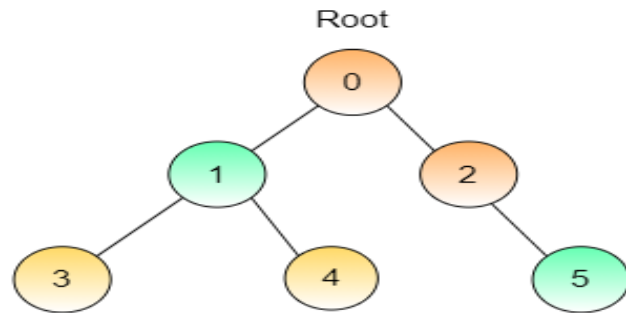
a) Remove the node at the top of the Stack i.e Node 0.

b) Add Node 0 to DFS traversal order.

c) Push the nodes adjacent to Node 0 that have not been visited, into the Stack.
i.e Node 1 & Node 2.



DFS Traversal Order

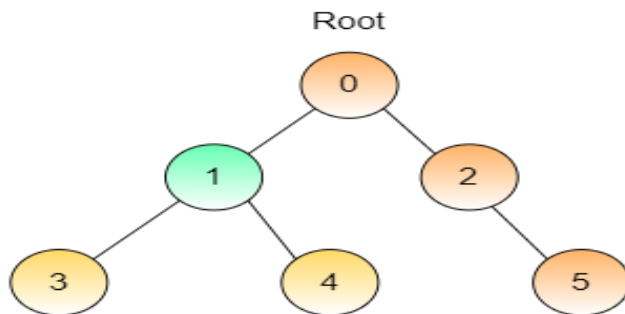
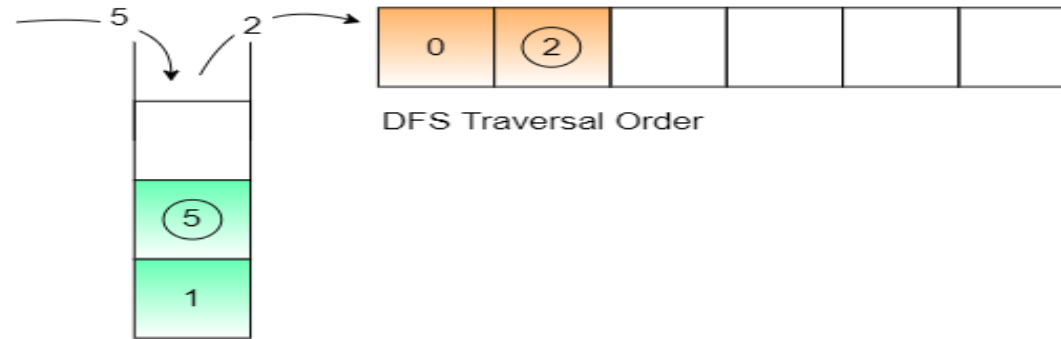


Step 3. While the Stack is not empty. Do the below

a) Remove the node at the top of the Stack i.e Node 2.

b) Add Node 2 to the DFS traversal order.

c) Push the nodes adjacent to Node 2 that have not been visited, into the Stack. i.e Node 5.

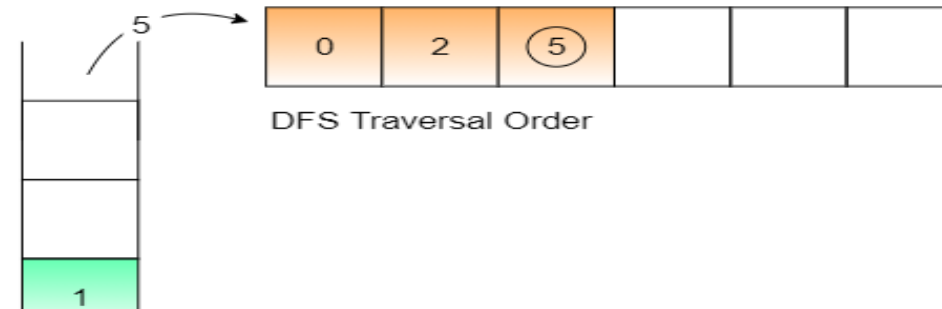


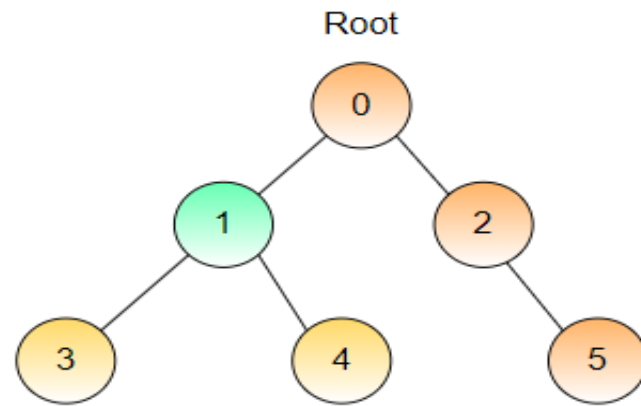
Step 4. While the Stack is not empty. Do the below

a) Remove the node at the top of the Stack i.e Node 5.

b) Add Node 5 to the DFS traversal order.

c) Push the nodes adjacent to Node 5 that have not been visited, into the Stack. i. **None**. Since the only node adjacent to Node 5 is Node 2 which has already been visited.



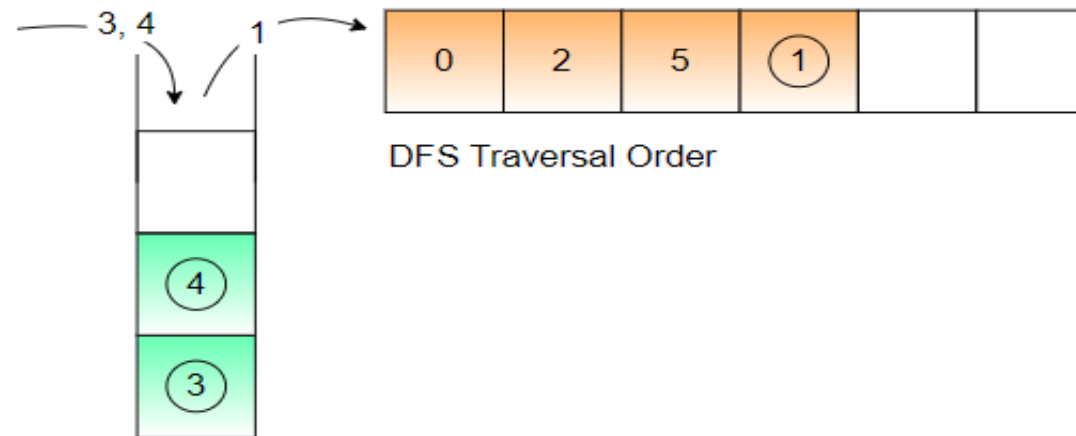


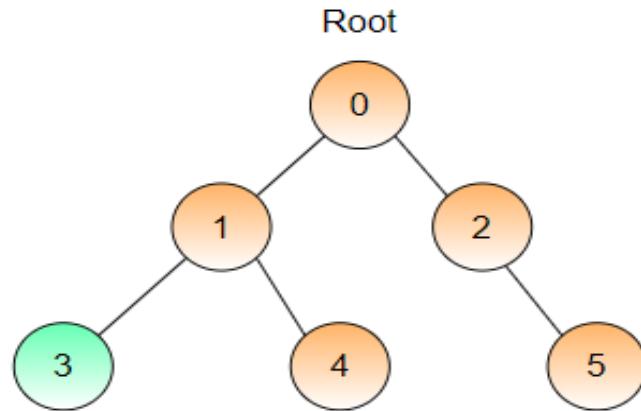
Step 5. While the Stack is not empty. Do the below

a) Remove the node at the top of the Stack i.e Node 1.

b) Add Node 1 to the DFS traversal order.

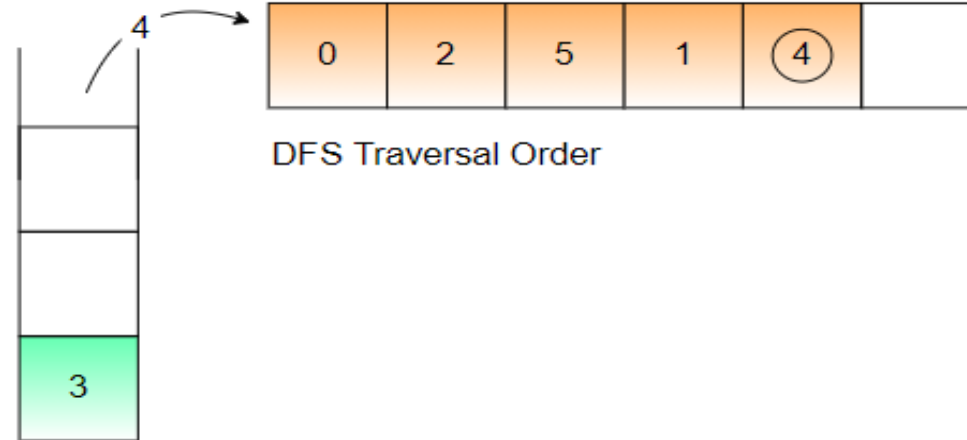
c) Push the nodes adjacent to Node 1 that have not been visited, into the Stack. i.e Node 3 and Node 4.

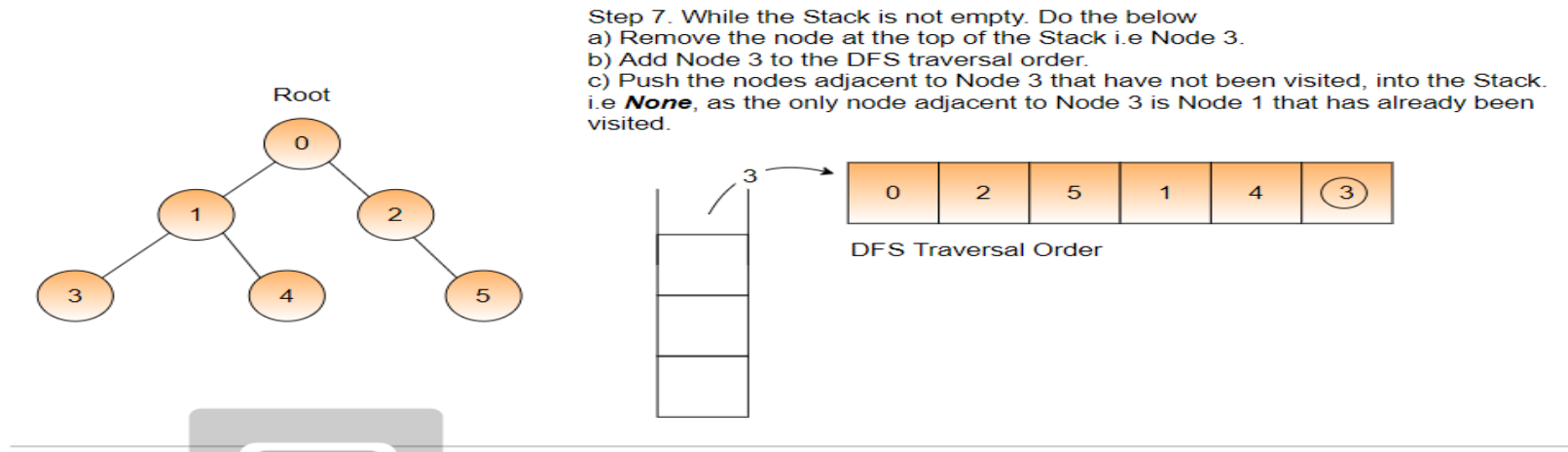




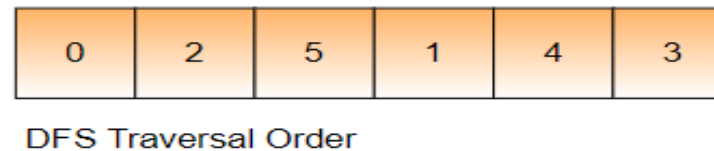
Step 6. While the Stack is not empty. Do the below

- Remove the node at the top of the Stack i.e Node 4.
- Add Node 4 to the DFS traversal order.
- Push the nodes adjacent to Node 4 that have not been visited, into the Stack. i.e **None**, as the only node adjacent to Node 4 is Node 1 that has already been visited.





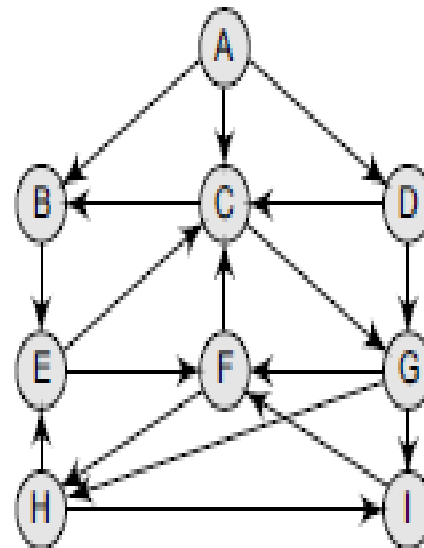
Stack is empty, thus all the nodes are visited in the DFS order.



EXAMPLE-2

Depth-first Search Algorithm:

Consider the graph and find the spanning tree,



Adjacency lists

A: B, C, D
B: E
C: B, G
D: C, G
E: C, F
F: C, H
G: F, H, I
H: E, I
I: F

Depth-first Search Algorithm:

- (a) Push H onto the stack.

STACK: H

- (b) Pop and print the top element of the STACK, that is, H. Push all the neighbours of H onto the stack that are in the ready state. The STACK now becomes

PRINT: H

STACK: E, I

- (c) Pop and print the top element of the STACK, that is, I. Push all the neighbours of I onto the stack that are in the ready state. The STACK now becomes

PRINT: I

STACK: E, F

- (d) Pop and print the top element of the STACK, that is, F. Push all the neighbours of F onto the stack that are in the ready state. (Note F has two neighbours, C and H. But only C will be added, as H is not in the ready state.) The STACK now becomes

PRINT: F

STACK: E, C

- (e) Pop and print the top element of the STACK, that is, C. Push all the neighbours of C onto the stack that are in the ready state. The STACK now becomes

PRINT: C

STACK: E, B, G

Depth-first Search Algorithm:

- (f) Pop and print the top element of the **STACK**, that is, G. Push all the neighbours of G onto the stack that are in the ready state. Since there are no neighbours of G that are in the ready state, no push operation is performed. The **STACK** now becomes

PRINT: G

STACK: E, B

- (g) Pop and print the top element of the **STACK**, that is, B. Push all the neighbours of B onto the stack that are in the ready state. Since there are no neighbours of B that are in the ready state, no push operation is performed. The **STACK** now becomes

PRINT: B

STACK: E

- (h) Pop and print the top element of the **STACK**, that is, E. Push all the neighbours of E onto the stack that are in the ready state. Since there are no neighbours of E that are in the ready state, no push operation is performed. The **STACK** now becomes empty.

PRINT: E

STACK:

Since the **STACK** is now empty, the depth-first search of G starting at node H is complete and the nodes which were printed are:

H, I, F, C, G, B, E

These are the nodes which are reachable from the node H.

Data structure used for storing graph or tree :

AdjacencyList

Data structure used for depth first search :

Stack

Time complexity of depth first search :

$O (V + E)$ for an adjacency list implementation of a graph or a tree. ‘**V**’ is the number of vertices and ‘**E**’ is the number of edges in a graph/tree.

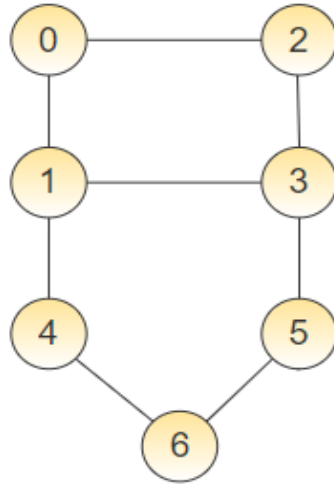
Applications of DFS

- **Detecting cycle in a graph**
- **Path Finding**
- **Solving puzzles with only one solution, such as mazes**

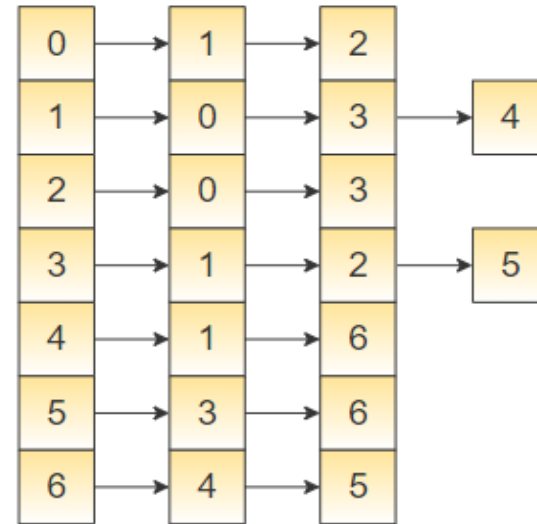
Adv and Disadv of BFS and DFS

<i>Advantages of Depth First Search</i>	<i>Advantages of Breadth First Search</i>
1. Simple to implement 2. Needs relatively small memory for storing the state-space	1. Guaranteed to find a solution (if one exists) 2. Depending on the problem, can be guaranteed to find an optimal solution
<i>Disadvantages of Depth First Search</i>	<i>Disadvantages of Breadth First Search</i>
1. Cannot find solution in all cases, and hence, can sometimes fail to find a solution 2. Not guaranteed to find an optimal solution can take a lot of time to find a solution	1. Memory management along with allocation is the key factor, and hence, more complex to implement 2. Needs a lot of memory for storing the state space if the search space has a high branching factor

Find the solution



DFS Traversal order for an un-weighted graph
 Starting node (0) : 0 1 3 2 5 6 4
 Starting node (3) : 3 1 0 2 4 6 5



Adjacency list representation for storing the graph

Uniform Cost search

- Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph.
- This algorithm comes into play when a different cost is available for each edge.
- The primary goal of the uniform-cost search is to find a path to the goal node which has the **lowest cumulative cost**. Uniform-cost search expands nodes according to their path costs from the root node.
- It can be used to solve any graph/tree where the optimal cost is in demand.
- A uniform-cost search algorithm is implemented by the **priority queue**.
- It gives maximum priority to the lowest cumulative cost. Uniform cost search is equivalent to BFS algorithm if the path cost of all edges is the same.

Uniform Cost search - Algorithm

function UNIFORM-COST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

frontier $\leftarrow$ a priority queue ordered by PATH-COST, with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the lowest-cost node in *frontier* */

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

 add *node*.STATE to *explored*

for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

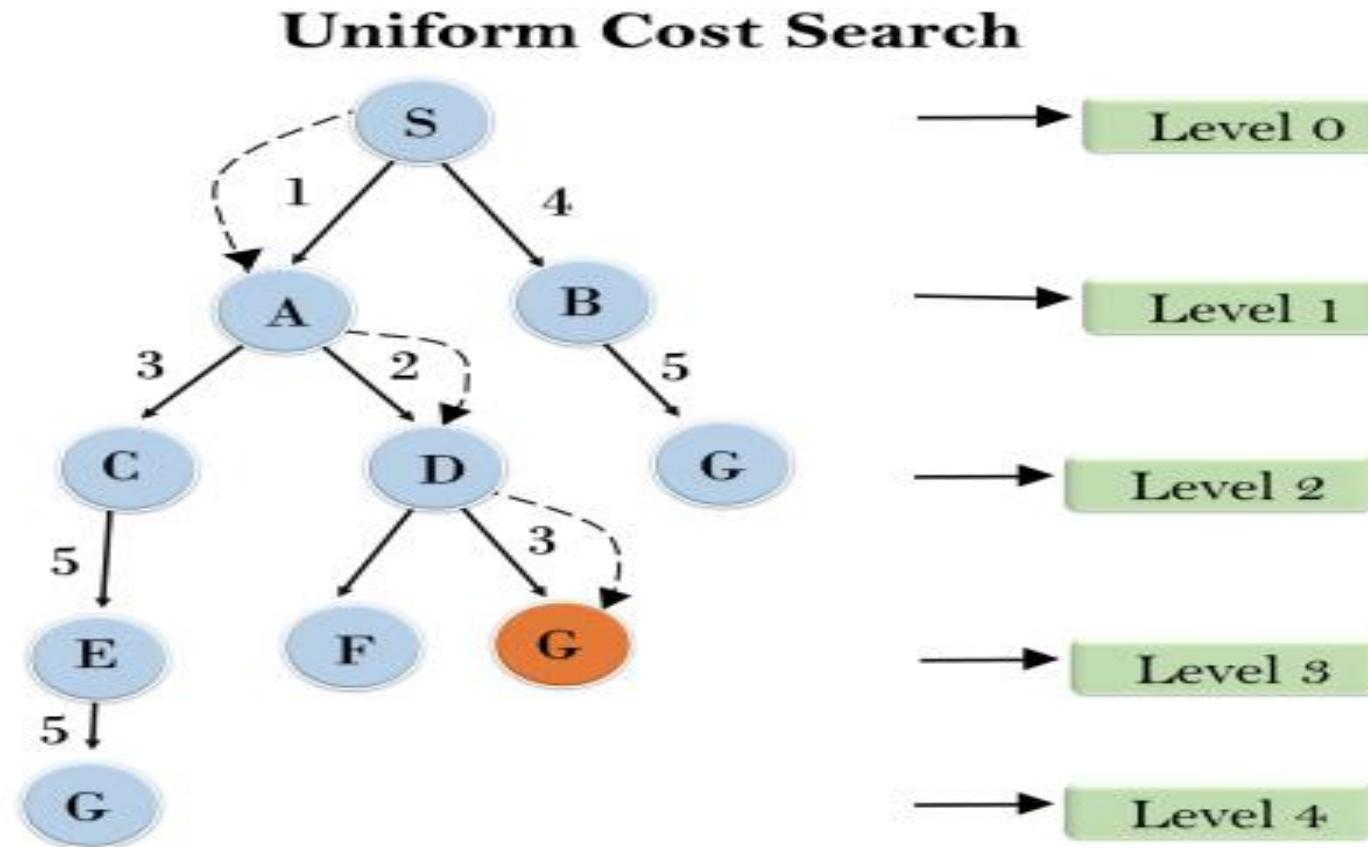
if *child*.STATE is not in *explored* or *frontier* **then**

frontier $\leftarrow$ INSERT(*child*, *frontier*)

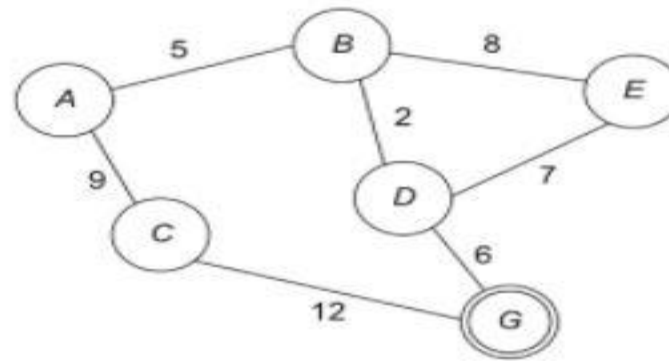
else if *child*.STATE is in *frontier* with higher PATH-COST **then**

 replace that *frontier* node with *child*

Uniform Cost search - Example



Uniform Cost search- Working



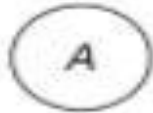
Uniform Cost search - Implementation

Step 1: Start from *A*

Expanded none

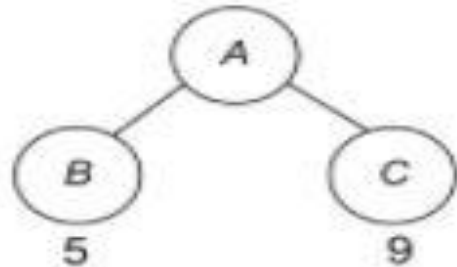
Step 2: Frontier

Expanded none



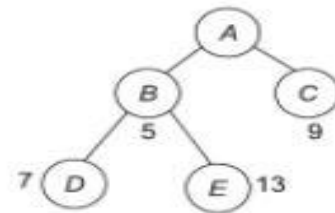
Step 3: Frontier: *B* and *C*

Expanded *A*



Uniform Cost search - Implementation

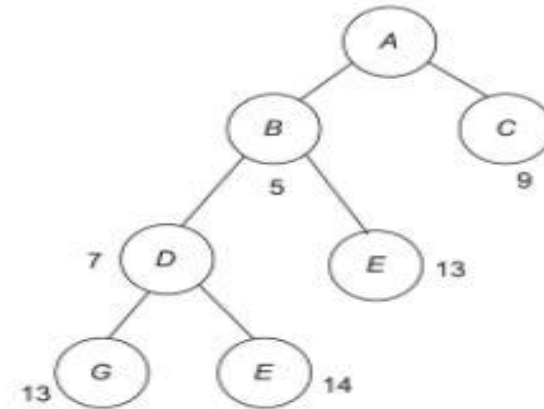
Step 4: Frontier: D, C, E (priority queue) Expanded A, B



Note: Remember the costs are added from the root till the node in consideration. Select the lowest, i.e., D.

Step 5: Frontier (C, E, E, G)

Expanded A, B, D



Active
Go to S

Uniform Cost search- Time and space

Completeness:

- YES, if step-cost $> \epsilon$ (small positive constant)

Time complexity:

- Assume C^* the cost of the optimal solution.
- Assume that every action costs at least ϵ
- Worst-case:

$$O(b^{C^*/\epsilon})$$

Space complexity:

- Idem to time complexity

Optimality:

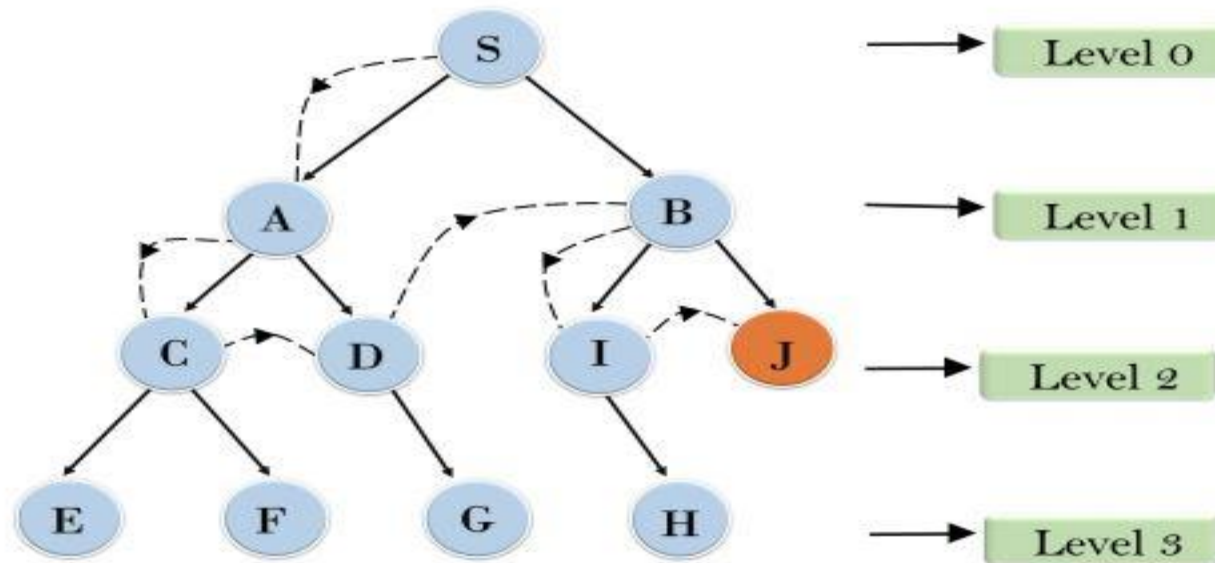
- nodes expanded in order of increasing path cost.
- YES, if complete.

Depth Limited Search

- A depth-limited search algorithm is similar to depth-first search with a predetermined limit.
- Depth-limited search can solve the drawback of the infinite path in the Depth-first search.
- In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

DLS

Depth Limited Search



DLS - Algorithm

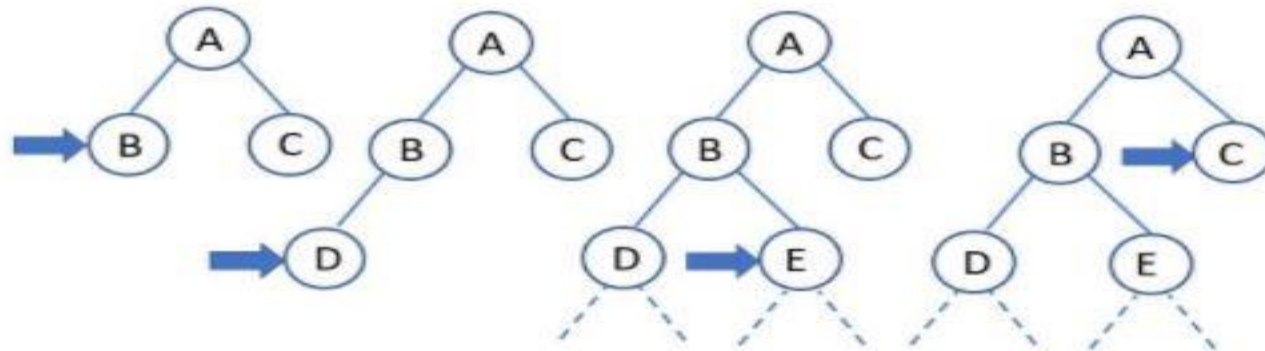
function DEPTH-LIMITED-SEARCH(*problem*, *limit*) **returns** a solution, or failure/cutoff
 return RECURSIVE-DLS(MAKE-NODE(*problem*.INITIAL-STATE), *problem*, *limit*)

function RECURSIVE-DLS(*node*, *problem*, *limit*) **returns** a solution, or failure/cutoff
 if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)
 else if *limit* = 0 **then return** *cutoff*
 else
 cutoff_occurred? $\leftarrow$ false
 for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**
 child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)
 result $\leftarrow$ RECURSIVE-DLS(*child*, *problem*, *limit* - 1)
 if *result* = *cutoff* **then** *cutoff_occurred?* $\leftarrow$ true
 else if *result* $\neq$ failure **then return** *result*
 if *cutoff_occurred?* **then return** *cutoff* **else return** failure

DLS - Limitation

- Standard failure value: It indicates that problem does not have any solution.
- Cutoff failure value: It defines no solution for the problem within a given depth limit.
- **Advantages:**
 - Depth-limited search is Memory efficient.
- **Disadvantages:**
 - Depth-limited search also has a disadvantage of incompleteness.
 - It may not be optimal if the problem has more than one solution

DLS - Working



DLS – Time and Space

- **Completeness**: DLS search algorithm is complete if the solution is above the depth-limit.
- **Time Complexity**: Time complexity of DLS algorithm is $O(b^{\ell})$.
- **Space Complexity**: Space complexity of DLS algorithm is $O(b \times \ell)$.
- **Optimal**: Depth-limited search can be viewed as a special case of DFS, and it is also not optimal even if $\ell > d$.

Comparison of Uninformed search strategies

TABLE 3.3 Comparison of Uninformed Search Techniques

<i>Search techniques</i>	<i>Complete</i>	<i>Optimal</i>	<i>Time complexity</i>	<i>Space complexity</i>
Breadth first search	Yes	Yes	$O(b^d)$	$O(b^d)$
Uniform cost	Yes	Yes	$O(b^d)$	$O(b^d)$
Depth first	No	No	$O(b^d)$	$O(bd)$
Depth limited	No	No	$O(b^l)$	$O(bl)$

Informed Search

- Informed Search algorithms have information on the goal state which helps in more efficient searching.
- This information is obtained by a function that estimates how close a state is to the goal state.
- Informed search algorithm uses the idea of heuristic, so it is also called Heuristic search.

Heuristic Search

- Add domain-specific information to select what is the best path to continue searching along
- Define a **heuristic function**, $h(n)$, that estimates the "goodness" of a node n . Specifically, $h(n)$ = estimated cost (or distance) of minimal cost path from n to a goal state.
- The term heuristic means "serving to aid discovery" and is an estimate, based on domain-specific information that is computable from the current state description, of how close we are to a goal

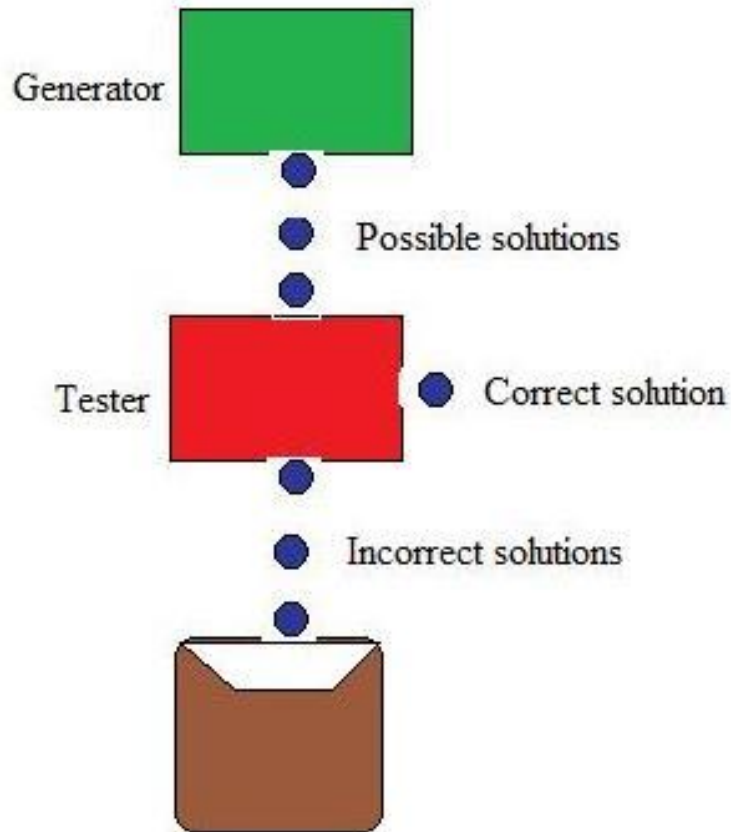
Generate and Test

- Generate-and-test search algorithm is a very simple algorithm that guarantees to find a solution if done systematically and there exists a solution.
- The generate and Test algorithm is a depth first search procedure because complete possible solutions are generated before test.
- This can be implemented states are likely to appear often in a tree
- It can be implemented on a search graph rather than a tree

Heuristic function

- Heuristic is a function which is used in Informed Search, and it finds the most promising path.
- It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal.
- The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time.
- Heuristic function estimates how close a state is to the goal. It is represented by $h(n)$, and it calculates the cost of an optimal path between the pair of states.
- The value of the heuristic function is always positive
- $h(n) \leq h^*(n)$
- **Here $h(n)$ is heuristic cost, and $h^*(n)$ is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.**

Generate and Test



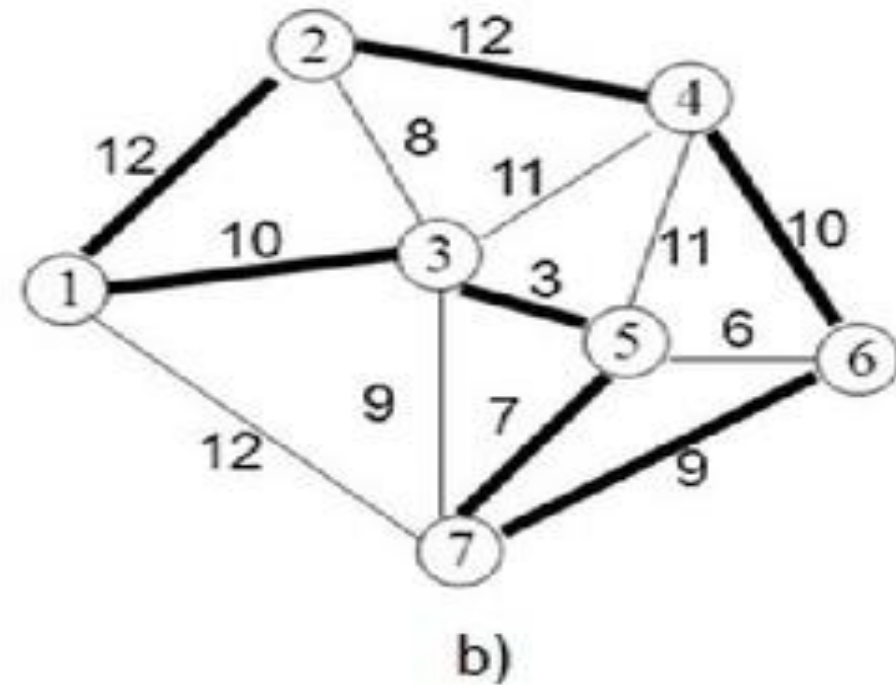
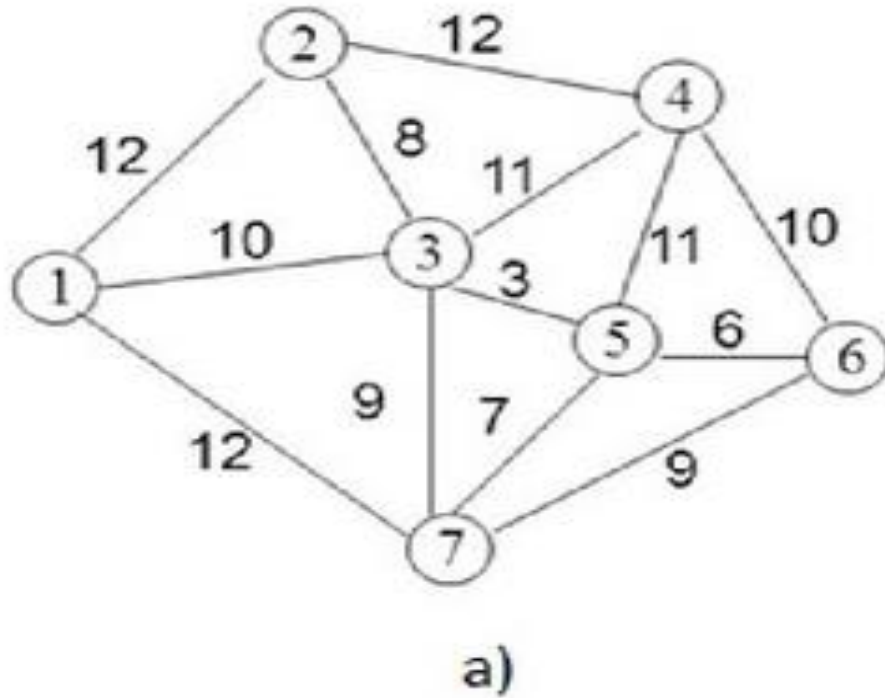
1. Generate a possible solution.
2. Test the solution.
3. If solution found THEN done ELSE return to step 1.

Generate and Test - Example

- The delivery robot must carry out a number of delivery activities, A, B, C, D, and E. Suppose that each activity happens at any of times 1, 2, 3, or 4
- $\{B \neq 3\}, (C \neq 2), (A \neq B), (B \neq C), (C < D), (A = D), (E < A), (E < B), (E < C), (E < D), (B \neq D)\}$

A –	1 4	4
B – 1 2 4	2	
C - 1 3 4	3	
D -	4	4
E		1

TSP - Example



1 - 3 - 5 - 7 - 6 - 4 - 2

Pure Heuristic Search:

- Pure heuristic search is the simplest form of heuristic search algorithms. It expands nodes based on their heuristic value $h(n)$. It maintains two lists, OPEN and CLOSED list. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.
- On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list. The algorithm continues until a goal state is found.
- In the informed search we will discuss two main algorithms which are given below:
- **Best First Search Algorithm(Greedy search)**
- **A* Search Algorithm**

Best First Search

- Best first search is a traversal technique that decides which node is to be visited next by checking which node is the most promising one and then check it.
- Best first search is an instance of graph search algorithm in which a node is selected for expansion based on evaluation function $f(n)$
- Best first search can be implemented a priority queue, a data structure that will maintain the fringe in ascending order of f values.

Best-first Search Algorithm (Greedy Search):

- Greedy best-first search algorithm always selects the path which appears best at that moment. It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search. Best-first search allows us to take the advantages of both algorithms. With the help of best-first search, at each step, we can choose the most promising node. In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e.
- $f(n) = g(n)$.
- Where, $h(n)$ = estimated cost from node n to the goal.
- The greedy best first algorithm is implemented by the priority queue.

Best First Search - Algorithm

Step 1: Place the starting node into the OPEN list.

Step 2: If the OPEN list is empty, Stop and return failure.

Step 3: Remove the node n , from the OPEN list which has the lowest value of $h(n)$, and places it in the CLOSED list.

Step 4: Expand the node n , and generate the successors of node n .

Step 5: Check each successor of node n , and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.

Step 6: For each successor node, algorithm checks for evaluation function $f(n)$, and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.

Step 7: Return to Step 2.

Adv and Disadv

Advantages:

Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.

This algorithm is more efficient than BFS and DFS algorithms.

Disadvantages:

It can behave as an unguided depth-first search in the worst case scenario.

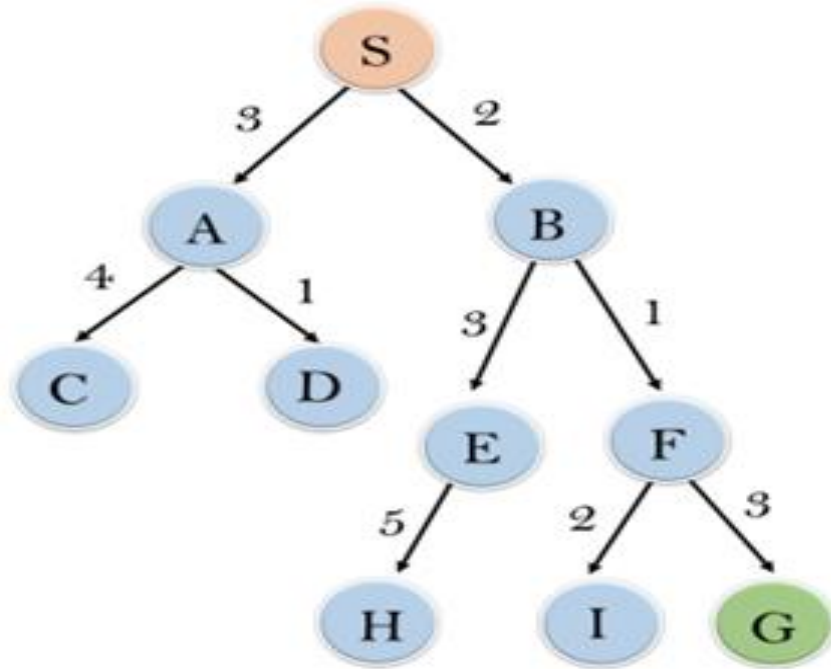
It can get stuck in a loop as DFS.

This algorithm is not optimal.

Applications of BFS

- Best-first search and its more advanced variants have been used in such applications as **games and web crawlers**.
- In a web crawler, each web page is treated as a node, and all the hyperlinks on the page are treated as unvisited successor nodes.

BFS - Example

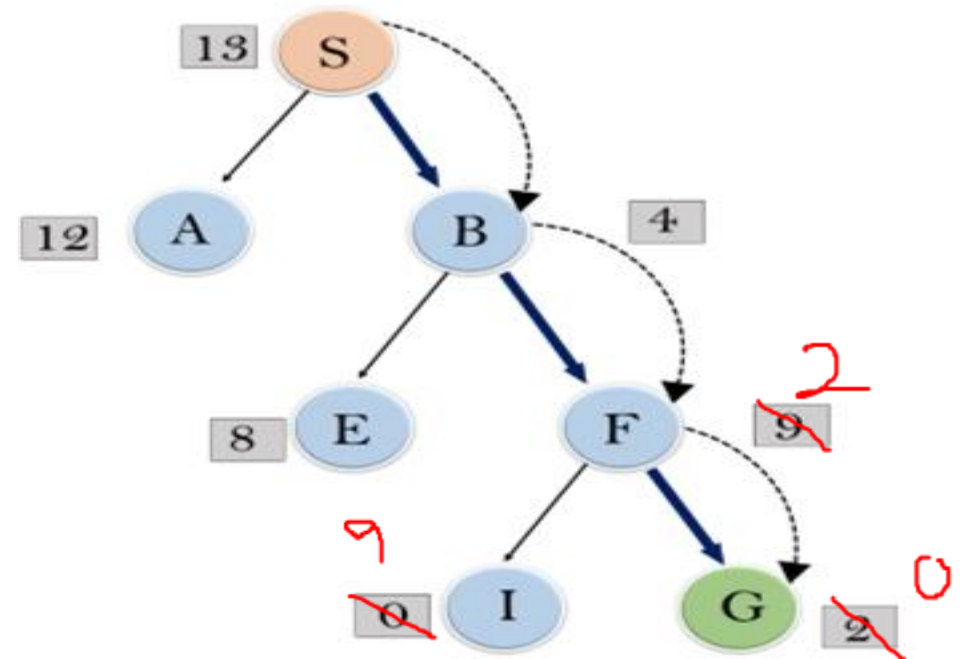


node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

Solution

- **Expand the nodes of S and put in the CLOSED list**
- **Initialization:** Open [A, B], Closed [S]
- **Iteration 1:** Open [A], Closed [S, B]
- **Iteration 2:** Open [E, F, A], Closed [S, B]
: Open [E, A], Closed [S, B, F]
- **Iteration 3:** Open [I, G, E, A], Closed [S, B, F]
: Open [I, E, A], Closed [S, B, F, G]
- Hence the final solution path will be:

S-----> B----->F-----> G



Time and Space complexity

Time Complexity: The worst case time complexity of Greedy best first search is $O(b^m)$.

Space Complexity: The worst case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.

Complete: Greedy best-first search is also incomplete, even if the given state space is finite.

Optimal: Greedy best first search algorithm is not optimal.

A* Search

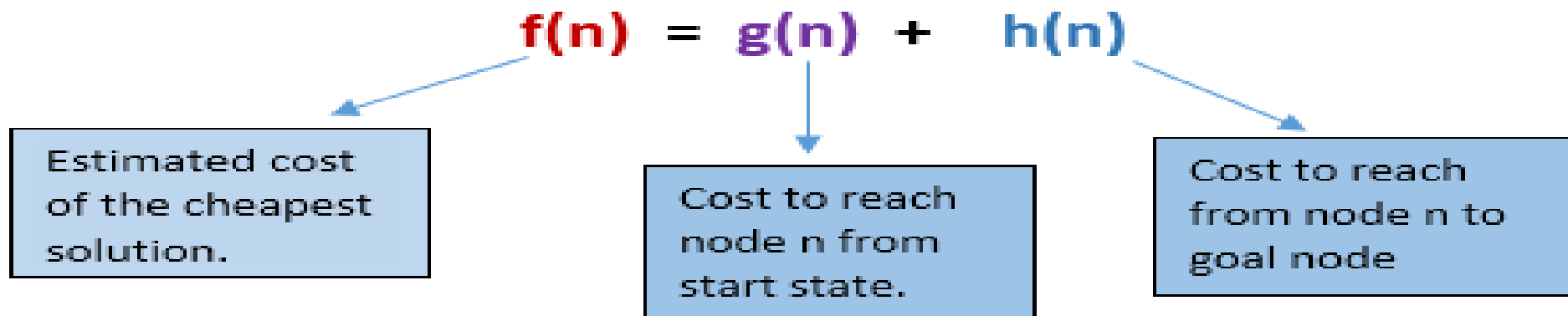
- It is an informed search algorithm, as it uses information about path cost and also uses heuristics to find the solution.
- To find the shortest path between nodes or graphs
- **A * algorithm** is a searching algorithm that searches for the shortest path between the *initial and the final state*
- It is an **advanced BFS** that *searches for shorter paths first rather than the longer paths.*
- A* is **optimal** as well as a **complete** algorithm

A* Search

- It calculates the cost, $f(n)$ (n being the neighboring node), to travel to all of the neighboring nodes, and then enters the node with the lowest value of $f(n)$.
- These values are calculated with the following formula:

$$f(n)=g(n)+h(n)$$

- $g(n)$ being the value of the shortest path from the start node to node n , and $h(n)$ being a heuristic approximation of the node's value.



Heuristic value $h(n)$

- A given heuristic function $h(n)$ is *admissible* if it never overestimates the real distance between n and the goal node.
- Therefore, for every node n , $h(n) \leq h^*(n)$
- $h^*(n)$ being the real distance between n and the goal node.
- A given heuristic function $h(n)$ is *consistent* if the estimate is always less than or equal to the estimated distance between the goal n and any given neighbor, plus the estimated cost of reaching that neighbor: $c(n,m) + h(m) \geq h(n)$
- $c(n,m)$ being the distance between nodes n and m . Additionally, if $h(n)$ is consistent, then we know the optimal path to any node that has been already inspected. This means that this function is *optimal*.

A* Search Algorithm

Step1: Place the starting node in the OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function ($g+h$), if node n is goal node then return success and stop, otherwise

Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n' , check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.

Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest $g(n')$ value.

Step 6: Return to **Step 2**.

Adv and Disadv

Advantages:

A* search algorithm is the best algorithm than other search algorithms.

A* search algorithm is optimal and complete.

This algorithm can solve very complex problems.

Disadvantages:

It does not always produce the shortest path as it mostly based on heuristics and approximation.

A* search algorithm has some complexity issues.

The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

Difference between BFS and A*

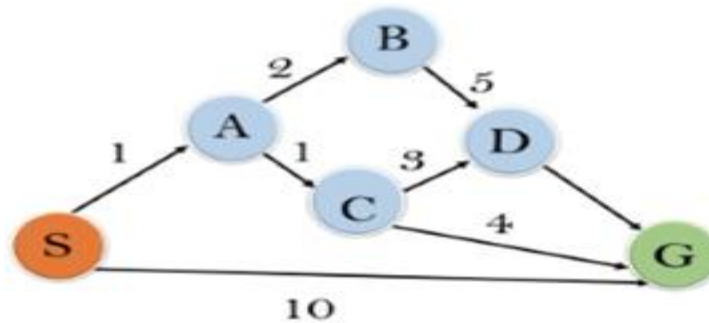
S	Parameters	Best-First Search	A* Search
1	Evaluation Function	The evaluation function for best-first search is $f(n) = h(n)$.	The evaluation function for A* search is $f(n) = h(n) + g(n)$.
2	Past Knowledge	This search algorithm does not involve past knowledge.	This search algorithm involves past knowledge.
3	Completeness	Best-first search is not complete.	A* search is complete.
4	Optimal	Best-first search is not optimal as the path found may not be optimal.	A* search is optimal as the path found is always optimal.
5	Time and Space Complexity	Its time complexity is $O(b^m)$ and space complexity can be polynomial. where b is the branching and m is the maximum depth of the search tree	Its time complexity is $O(b^m)$ and space complexity is also $O(b^m)$. where b is the branching and m is the maximum depth of the search tree
6	Memory	It requires less memory.	It requires more memory.
7	Type of nodes kept	It keeps all the fringe or border nodes in the memory while searching.	It keeps all the nodes in the memory while searching.

Applications of A*

- It is commonly used in web-based maps and games to find the shortest path at the highest possible efficiency.
- A* is used in many artificial intelligence applications, such as search engines.
- It is used in other algorithms such as the Bellman-Ford algorithm to solve the shortest path problem.

Example

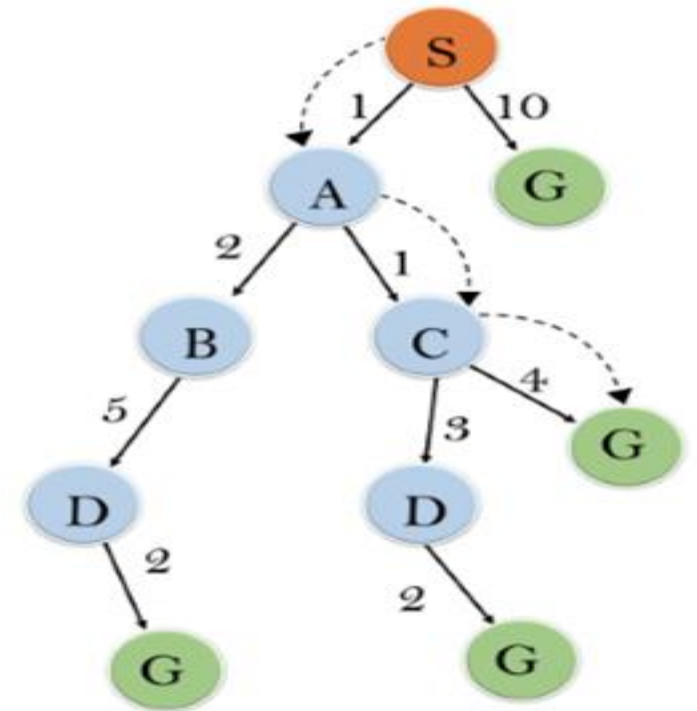
- In this example, we will traverse the given graph using the A* algorithm. The heuristic value of all states is given in the below table so we will calculate the $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach any node from start state.
- Here we will use OPEN and CLOSED list.



State	$h(n)$
S	5
A	3
B	4
C	2
D	6
G	0

Solution

- **Initialization:** $\{(S, 5)\}$
- **Iteration1:** $\{(S \rightarrow A, 4), (S \rightarrow G, 10)\}$
- **Iteration2:** $\{(S \rightarrow A \rightarrow C, 4), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$
- **Iteration3:** $\{(S \rightarrow A \rightarrow C \rightarrow G, 6), (S \rightarrow A \rightarrow C \rightarrow D, 11), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$
- **Iteration 4** will give the final result, as
 $S \rightarrow A \rightarrow C \rightarrow G$ it provides the optimal path with cost 6.



Points to remember:

- A* algorithm returns the path which occurred first, and it does not search for all remaining paths.
- The efficiency of A* algorithm depends on the quality of heuristic.
- A* algorithm expands all nodes which satisfy the condition $f(n) \leq l_i$

Complete: A* algorithm is complete as long as:

- Branching factor is finite.
- Cost at every action is fixed.

Optimal: A* search algorithm is optimal if it follows below two conditions:

Admissible: the first condition requires for optimality is that $h(n)$ should be an admissible heuristic for A* tree search. An admissible heuristic is optimistic in nature.

Consistency: Second required condition is consistency for only A* graph-search.

- If the heuristic function is admissible, then A* tree search will always find the least cost path.

Time and Space complexity

- **Time Complexity:** The time complexity of A* search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d . So the time complexity is $O(b^d)$, where b is the branching factor.
- **Space Complexity:** The space complexity of A* search algorithm is $O(b^d)$

Example-2

- Given an initial state of a 8-puzzle problem and final state to be reached-

2	8	3
1	6	4
7		5

Initial State

1	2	3
8		4
7	6	5

Final State

Find the most cost-effective path to reach the final state from initial state using A* Algorithm.
Consider $g(n)$ = Depth of node and $h(n)$ = Number of misplaced tiles.

Solution

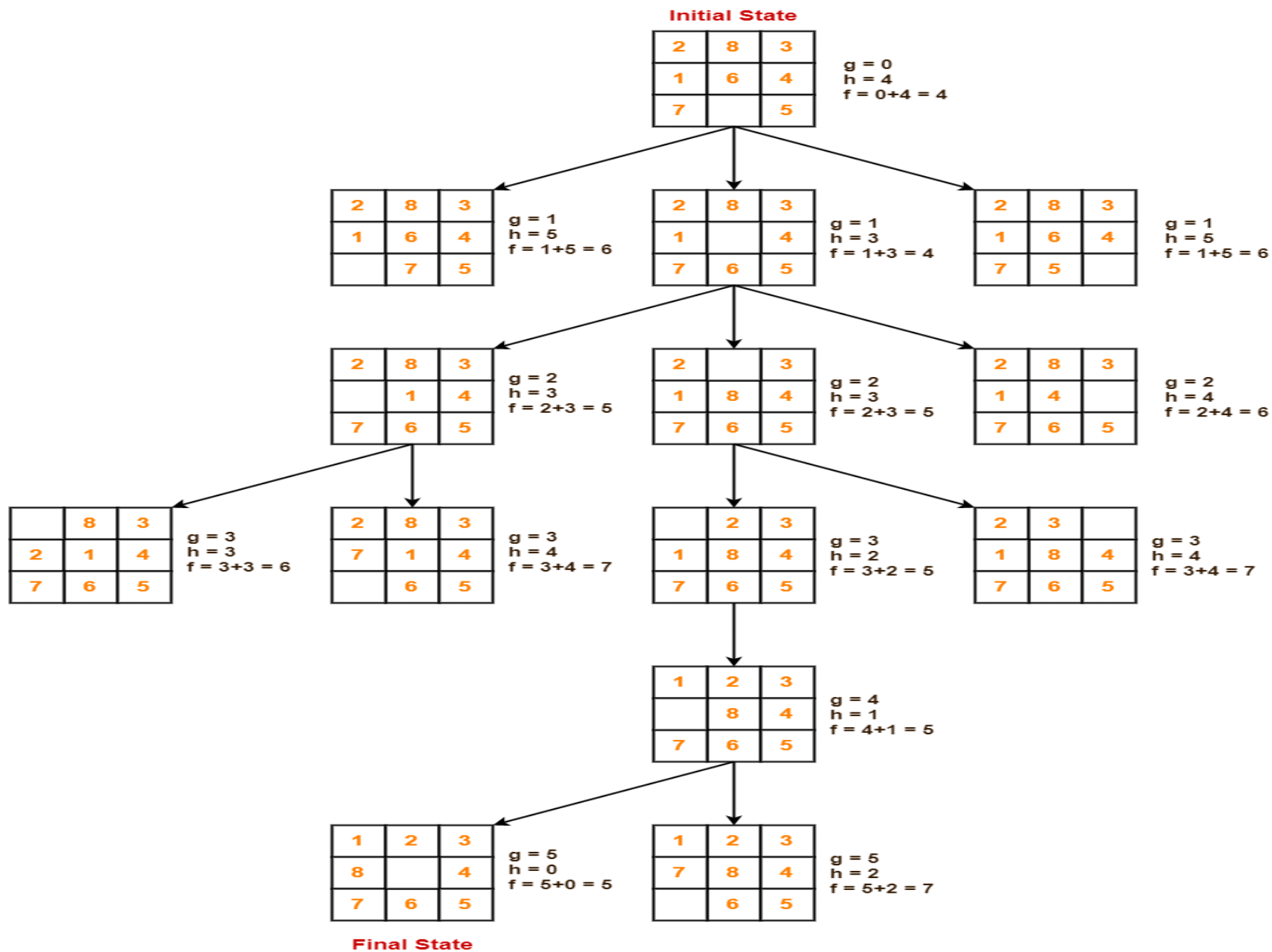
- A* Algorithm maintains a tree of paths originating at the initial state.
- It extends those paths one edge at a time.
- It continues until final state is reached.

2	8	3
1	6	4
7		5

Initial State

1	2	3
8		4
7	6	5

Final State



References

- *Parag Kulkarni, Prachi Joshi, Artificial Intelligence –Building Intelliegent Systems, 1st ed., PHI learning,2015*
- *Stuart J. Russell, Peter Norwig , Artificial Intelligence –A Modern approach, 3rd Pearson Education, 2016*
- Kevin Night and Elaine Rich, Nair B., “Artificial Intelligence (SIE)”, Mc Graw Hill- 2008.

Thank You